AI DRIVEN PRIVACY PRESERVING DIGITAL ECOSYSTEM FOR MODERNIZING SRI LANKA'S PADDY SUPPLY CHAIN

INDIVIDUAL THESIS

Kalehe Singhage Ishara Chamudi

(IT22147950)

Individual component: AI Powered Farmer Miller Marketplace

Group ID: R26-SE-007

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

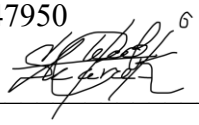
DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

*Student name: Kalehe Singhage Ishara Chamudi

Student ID: IT22147950

Signature:  Date: 09/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Dr. Mahima Weerasinghe		
Co-supervisor: Mr. Eshan Weerasinghe		

ABSTRACT

The Sri Lankan paddy supply chain faces challenges related to fragmented market information, limited farmer-miller coordination, and difficulties in identifying suitable trading partners and negotiating prices. These challenges can reduce the accessibility of relevant market information and make trading decisions more difficult for farmers and millers. This research addresses the need for an accessible digital marketplace that supports partner discovery, provides contextual agricultural information, and assists price-related decision-making while recognizing the importance of responsible data handling. The aim of this individual research component is to design and develop an **AI-powered farmer-miller marketplace** as part of the **Digital Goviya** ecosystem. A design and development approach was adopted to implement a mobile application using React Native and Expo, an Express.js and python backend, and MongoDB. The proposed system integrates a rule-based farmer-miller matching mechanism with AI-assisted negotiation to support agricultural trading decisions. The multi-agent negotiation component employs separate farmer and miller agents that generate offers and counteroffers within a controlled negotiation workflow, supported by rule-based validation and negotiation-state management. Federated learning is explored through a price-reference prediction component that simulates collaborative model training across district-based clients while aggregating model parameters into a global model. Additionally, a retrieval-augmented generation (RAG) assistant combines multilingual semantic retrieval with large language model generation to provide contextual information related to the marketplace and agricultural price mechanisms. Together, these components demonstrate how intelligent matching, multi-agent negotiation, federated learning, and knowledge-grounded assistance can be integrated into a digital marketplace prototype. The proposed solution provides a foundation for further research into transparent farmer-miller interactions, AI-assisted decision-making, and privacy-aware digital agricultural services in Sri Lanka.

Keywords: Digital agriculture, farmer-miller marketplace, retrieval-augmented generation, federated learning, Multi agent negotiation

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my research supervisors, Dr. Mahima Weerasinghe and Mr. Eishan Weerasinghe, for their guidance, feedback, and support throughout the development of my individual research component. Their advice helped me refine the research direction, consider appropriate technical approaches, and improve the design and implementation of the proposed marketplace.

I am also grateful to the academic staff of the Sri Lanka Institute of Information Technology for the knowledge and learning opportunities provided throughout my degree program. I would like to acknowledge my project team members for their collaboration in developing the broader Digital Goviya ecosystem, particularly the discussions and coordination required to position the marketplace component within the overall solution.

Finally, I would like to thank my family and friends for their encouragement and understanding during the research and development process. Their support helped me continue working towards the completion of this research.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 3

1.3 Problem statement 6

1.4 Research gap 9

1.5 Aim and objectives 11

1.5.1 Research Aim 11

1.5.2 Specific Objectives 12

1.5.3 Alignment Between Objectives and Evaluation 13

1.6 Research questions or hypotheses 14

1.7 Scope limitations and assumptions 16

1.7.1 Research Scope 16

1.7.2 Limitations 17

1.7.3 Research Assumptions 19

1.8 Individual contribution 20

1.9 Thesis structure 22

2 LITERATURE REVIEW AND RELATED WORK 24

2.1 Review method and scope 24

2.2 Domain and theoretical foundations 25

2.3 Related research 30

2.4 Existing systems products and techniques 33

2.5 Gap synthesis and conceptual framework 37

3 RESEARCH METHODOLOGY 42

3.1 Research design 42

3.2 Data participants or study materials 43
3.3 Experimental or study procedure 47
3.4 Measures and evaluation metrics 51
3.5 Data analysis 53
3.6 Ethics privacy security and responsible AI 54
3.7 Reproducibility and research artefacts 56
4 PRODUCT AND SOLUTION DESIGN 58
4.1 Requirements 58
4.2 Component architecture within the complete solution 61
4.3 Design alternatives and decisions 64
4.4 Detailed design 68
4.5 Standards and professional practice 73
5 IMPLEMENTATION INTEGRATION AND TESTING 75
5.1 Development environment and technology stack 75
5.2 Component implementation 77
5.3 Integration with the other components 79
5.4 Verification and software testing 82
5.5 Deployment and operation 84
6 RESULTS AND VALIDATION 87
6.1 Research results 87
6.2 Product and technical results 89
6.3 User or stakeholder validation 91
6.4 Integration results 91
6.5 Summary of findings 92
7 DISCUSSION 94
7.1 Interpretation of findings 94

7.2 Comparison with literature baselines and existing solutions 96
7.3 Research contribution..... 99
7.4 Product contribution..... 100
7.5 Limitations and threats to validity..... 102
7.6 Implications..... 105
8 CONCLUSION AND FUTURE DIRECTIONS 107
8.1 Conclusion 107
8.2 Achievement of objectives..... 108
8.3 Individual research and product contributions..... 109
8.4 Future directions..... 111
REFERENCES..... 114
GLOSSARY..... 115
APPENDICES 117
Appendix A Research instruments or extended results..... 117
Appendix B Academic integrity and AI use disclosure 118

LIST OF FIGURES

Figure 1 : Position of the Marketplace within the Digital Goviya Ecosystem 3
Figure 2 : Research and Development Methodology 43
Figure 3 : Paddy Marketing Board price information 44
Figure 4 : Main source PDF for RAG assistant 46
Figure 5 : Analysis of Negotiation Outcomes 50
Figure 6 : Analysis of Conversation Depth 50
Figure 7 : Marketplace Evaluation measures 52
Figure 8 : Digital Goviya Overall System Architecture 62
Figure 9 : AI-Powered Farmer-Miller Marketplace System Diagram 63
Figure 10 : RAG Architecture 66
Figure 11 : Federated Learning Simulation workflow 67
Figure 12 : Negotiation Agents Orchestration 71
Figure 13 : Matching Life cycle 72
Figure 14 : Farmer and Miller Platforms of Digital Goviya 101

LIST OF TABLES

Table 1 : Comparison of relevant technical approaches 35
Table 2 : Recommended framework for comparing existing systems 36
Table 3 : Mapping identified gaps to Marketplace features and evaluation 40
Table 4 : Functional Requirement 59
Table 5 : Non-Functional Requirements 60
Table 6 : Technology Stack 76
Table 7 : Internal Communication Paths within component 80
Table 8 : Functional Test Cases 84
Table 9 : Overall Comparison 99
Table 10 : Table of Treats 104
Table 11 : Achievement of research objectives 109

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
FL	Federated Learning
FedAvg	Federated Averaging
FAISS	Facebook AI Similarity Search
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
LLM	Large Language Model
MAE	Mean Absolute Error
ML	Machine Learning
NLP	Natural Language Processing
PMB	Paddy Marketing Board
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
UI	User Interface

1 INTRODUCTION

1.1 Overall project context

Sri Lanka's paddy sector plays a significant role in the country's agricultural economy and food supply system. However, the modernization of the paddy supply chain requires more intelligent, secure, and privacy-preserving digital technologies capable of supporting farmers and other stakeholders across different stages of the agricultural and market ecosystem. Existing technological approaches generally address individual aspects of the paddy supply chain, such as crop monitoring, yield prediction, price forecasting, traceability, or online marketplaces, without providing an integrated ecosystem that combines intelligent decision-making, privacy protection, secure transactions, and stakeholder interaction.

The proposed research, "AI Driven Privacy Preserving Digital Ecosystem for Modernizing Sri Lanka's Paddy Supply Chain," addresses this gap through an integrated ecosystem consisting of four complementary components. The first component focuses on developing a Digital Dashboard for paddy farming, combining real-time data, deep learning, Vision Transformers, Retrieval-Augmented Generation (RAG), and Graph Neural Networks (GNNs) to support yield prediction, crop image analysis, and knowledge-augmented agricultural advisory services. The research specifically identifies the lack of systems that integrate multimodal agricultural inputs such as images, soil, weather, and farm metadata, as well as the limited application of Digital Dashboard, RAG, and GNNs within the Sri Lankan paddy farming context.

The second component addresses the uncertainty associated with paddy price forecasting. Conventional statistical and machine-learning approaches can predict prices, but they provide limited understanding of the underlying causes of price changes. Therefore, the proposed component combines Transformer-based time-series forecasting with Causal Analysis Learning, Large Language Model reasoning.

The third component focuses on establishing privacy-preserving and trustworthy supply-chain traceability. Although blockchain can provide tamper-resistant records for tracking agricultural products, conventional blockchain-based approaches may expose sensitive information about farmers and other participants. To address this limitation, the proposed ecosystem incorporates Zero-Knowledge Proofs (ZKPs), Decentralized Identity (DID), smart contracts, and AI-based fraud detection using Graph Machine Learning. These technologies are intended to enable stakeholders to verify relevant agricultural and transaction information without unnecessarily exposing sensitive data, while also supporting secure identities and the detection of fraudulent or collusive behaviour within the supply chain.

The fourth component introduces an AI-powered farmer-miller marketplace designed to improve market participation and decision-making while preserving stakeholder privacy. Unlike conventional agricultural marketplaces that primarily display prices, the proposed approach incorporates RAG-based LLM assistants, Federated Learning based reference price prediction, secure matching, and multi-agent negotiation. Federated Learning enables AI models to benefit from distributed farmer data without requiring the underlying private data to be centrally shared. Secure matching mechanisms can further protect sensitive information during buyer-seller interactions, while AI agents can support automated negotiation between farmers and millers.

Collectively, these four components establish a unified AI-driven, privacy-preserving digital ecosystem for Sri Lanka's paddy supply chain. The ecosystem aims to connect farm-level intelligence, market forecasting, secure and privacy-preserving traceability, and intelligent farmer-miller interactions within a common technological framework. By integrating Artificial Intelligence, Digital Dashboard, XGBoost-based forecasting, Causal Anaysis Learning, RAG, Graph Neural Networks, Blockchain, Zero-Knowledge Proofs, Decentralized Identity, Federated Learning, secure matching, and multi-agent systems, the research seeks to move beyond isolated agricultural

applications toward a more comprehensive digital ecosystem capable of supporting informed decision-making, trustworthy transactions, privacy protection, and efficient stakeholder interaction across the paddy supply chain.

1.2 Individual component context

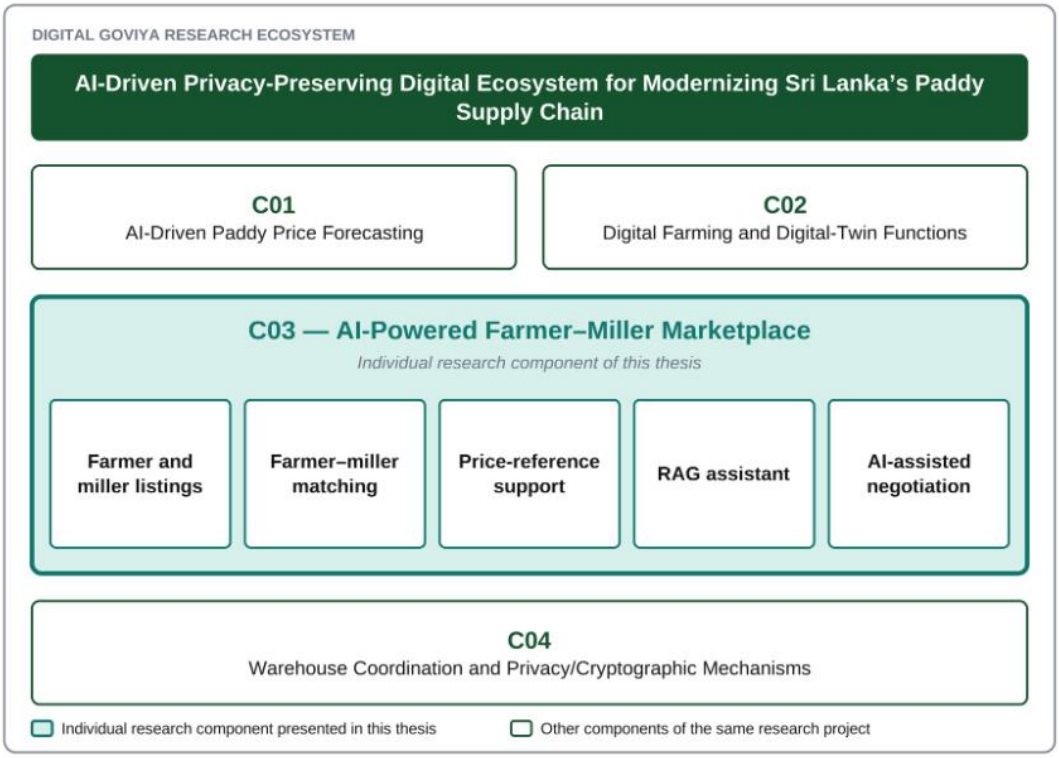


Figure 1 : Position of the Marketplace within the Digital Goviya Ecosystem

The AI-Powered Farmer-Miller Marketplace with Privacy and Negotiation is the third-party-independent marketplace component of the collaborative Digital Goviya research ecosystem. Its primary purpose is to facilitate digital interactions between paddy farmers who offer harvests for sale and rice millers who publish purchasing requirements. The component extends a conventional listing-based marketplace by incorporating AI-assisted decision support, a price-reference prediction mechanism, a weighted matching algorithm, and a multi-agent negotiation process.

In a conventional digital marketplace, users may be able to publish products or requirements and contact potential trading partners. Although this provides a digital communication channel, users may still need to determine whether an offer is reasonable, identify the most suitable partner, and negotiate a mutually acceptable price without structured assistance. The proposed marketplace addresses these challenges by connecting listing management with intelligent support mechanisms.

The marketplace is designed around two principal user roles, farmers and millers. Farmers can create and manage harvest listings containing relevant information such as paddy type, quantity, location, and expected selling price. Millers can create purchasing demands that represent their paddy requirements and buying preferences. These records provide the information required for the marketplace to compare available supply with purchasing demand.

The marketplace follows a transaction-support process that begins with the creation of a harvest or demand listing. The system can then estimate a reference price for a farmer's harvest, identify potential trading partners, facilitate match requests, and support negotiation between the two parties. When the negotiation process reaches an agreement, the application records the agreement-related status. The actual sale or fulfilment status can be managed separately, allowing the system to distinguish between reaching a negotiated agreement and confirming that a physical transaction has been completed.

Four principal technical features form the research focus of this component.

Retrieval-Augmented Generation (RAG) Market Advisory Assistant. The marketplace incorporates a question-answering assistant intended to help users access relevant marketplace information and curated knowledge concerning Paddy Marketing Board (PMB) purchasing information, applicable procedures, and system functionality. The assistant combines semantic retrieval with large language model generation. The implementation uses a multilingual sentence embedding model, a FAISS vector index, and the OpenAI GPT-4.1-mini model to retrieve relevant knowledge and generate contextual answers. This approach is intended to make

relevant information easier to access, including for users who interact in English, Sinhala, or Singlish. Its reliability depends on the coverage and quality of the supplied knowledge base and the accuracy of the retrieval and generation process.

Federated Learning Style Reference Price Prediction. The marketplace includes a farmer expected price-prediction mechanism intended to provide a reference value when a farmer creates a harvest listing. The implemented prototype uses a simplified Federated Averaging (FedAvg) simulation with a linear model based on scikit-learn's SGDRegressor. Four simulated district clients participate in model training, and model parameters are averaged over training rounds to produce a global model. The resulting model can generate a reference price from the supported input features, including district, paddy type, season, and quantity. In the current implementation, the training data is synthetic and the federated process is simulated within a single environment. Therefore, the output should be interpreted as an experimental reference estimate rather than an official PMB price or a validated prediction of the actual market price.

Weighted Farmer-Miller Matching. The marketplace implements a rule-based scoring algorithm to rank potential trading partners. The algorithm evaluates compatibility using district, paddy type, price closeness, and quantity suitability. In the implemented scoring scheme, district contributes up to 40 points, paddy type contributes 30 points, price closeness contributes 20 points, and quantity suitability contributes 10 points. The resulting score is used to rank candidates and communicate their relative compatibility to the user. Although the matching mechanism is described as AI-assisted within the wider system, the implemented pair-matching score is a weighted rule-based algorithm. The predicted reference price is one of its inputs.

Multi-Agent AI Negotiation System. The marketplace incorporates two software agents that represent the farmer and the miller during automated price negotiation. The implemented system uses a Python-based orchestration service integrated with OpenAI’s GPT model for AI-powered negotiation, and custom validation rules. The agents operate sequentially, proposing actions such as accepting an offer, making a counter-offer, or rejecting a proposal. The validation layer enforces the applicable private price limits and other negotiation constraints before decisions are accepted by

the system. This hybrid design combines language-model-generated proposals with deterministic rules intended to prevent invalid offers. The mechanism supports application-level separation of the parties' negotiation limits, but it should not be interpreted as cryptographic privacy or a guarantee against every possible information leak.

The four features address different stages of market decision-making. The RAG assistant supports knowledge access, price prediction supplies a reference indicator, matching narrows the search for potential trading partners, and negotiation supports structured bargaining. These functions are integrated with the marketplace's underlying application services, database, authentication mechanisms, and user interface.

The implementation uses an Expo and React Native mobile application, a Node.js and Express backend, MongoDB for persistent data storage, and Python services for selected AI and machine-learning functions. The Node.js backend manages application requests and coordinates with the relevant Python processes or services. Authentication and role-based access controls support the separation of farmer and miller activities.

The individual research contribution is therefore not limited to implementing four separate algorithms. It involves integrating them into a working marketplace workflow in which users can create listings, obtain reference information, discover compatible trading partners, submit match requests, negotiate, and manage transaction-related statuses. This integration provides the basis for investigating the practical usefulness, limitations, and potential future development of AI-assisted digital trading within Sri Lanka's paddy supply chain.

1.3 Problem statement

Paddy farmers and rice millers require reliable ways to exchange information, identify suitable trading partners, and agree on prices. However, traditional trading practices

can depend heavily on informal communication, local relationships, and manually conducted negotiations. When information about available supply, purchasing demand, and expected prices is fragmented, both parties may experience difficulty evaluating trading opportunities efficiently.

Farmers may not have sufficient information to assess whether a proposed selling price is reasonable or whether another buyer would provide a more suitable opportunity. They may also spend time contacting multiple millers to identify a buyer whose purchasing requirements match the available harvest. Millers face a corresponding challenge when searching for suitable suppliers whose paddy type, quantity, location, and expected price meet their requirements. These difficulties can increase the effort required to arrange transactions and limit the ability of both parties to make informed decisions.

The availability of digital marketplaces can improve access to trading opportunities, but listing and search functionality alone may not resolve these issues. A marketplace that does not provide suitable price guidance can leave users uncertain about how to evaluate offers. A marketplace that does not rank potential partners according to relevant requirements can require users to examine many unsuitable listings. Similarly, a marketplace that relies entirely on manual bargaining may not provide structured support for managing offers and counter-offers.

Access to useful market knowledge presents another challenge. Information concerning purchasing procedures, marketplace operations, and relevant market guidance may be distributed across different documents and information sources. Users who have difficulty locating or interpreting such information may struggle to make confident trading decisions. A conversational assistant can improve accessibility, but an unconstrained language model may generate unsupported statements. Consequently, market-related assistance requires a mechanism for retrieving relevant knowledge and grounding generated responses in the available information.

Data privacy is also an important consideration. Information about harvest quantities, expected selling prices, purchasing requirements, and negotiation limits can have commercial value. Exposing such information to the other party prematurely may influence bargaining behavior or reduce a participant's negotiating position. At the same time, predictive analytics can benefit from learning patterns across multiple data sources. This creates a design challenge, how can useful predictive information be generated while reducing unnecessary central exposure of sensitive data. Federated learning provides a possible approach to collaborative model training, although a simulated implementation does not by itself establish production-level privacy protection.

Finally, manual price negotiation can be time-consuming and difficult to structure consistently. Farmers and millers may have different expectations and limits, and each party needs to evaluate proposals without automatically revealing its private reservation price. A negotiation-support mechanism must therefore balance flexibility in proposing offers with explicit constraints that prevent invalid decisions.

The central problem addressed by this individual research is the lack of an integrated, AI-assisted digital marketplace prototype that combines market knowledge retrieval, reference-price estimation, buyer-seller matching, and structured negotiation support for Sri Lankan paddy trading while considering the protection of commercially sensitive information.

Accordingly, this research investigates how these mechanisms can be designed and integrated into a usable farmer-miller marketplace, and how the resulting prototype can be evaluated in terms of functional correctness, predictive performance, matching behavior, negotiation outcomes, and relevant software quality attributes.

The research does not assume that implementing these mechanisms automatically guarantees higher farmer income, eliminates intermediaries, or ensures fair market prices. Such outcomes require appropriate real-world data and field evaluation. Instead, the study focuses on the design, implementation, and technical evaluation of

a prototype intended to support more informed and structured interactions between farmers and millers.

1.4 Research gap

Existing research in digital agriculture demonstrates the potential of AI, machine learning, distributed computing, and digital platforms to improve agricultural decision-making and supply-chain coordination. However, the relevance of these technologies depends on the particular problem being addressed, the quality of the data used, and the extent to which individual techniques are integrated into practical workflows.

One area of research focuses on AI-powered agricultural advisory systems. Retrieval-Augmented Generation (RAG) provides a method for combining information retrieval with language-model generation, allowing a system to produce responses informed by a selected knowledge base. Such systems can improve access to specialized information. Nevertheless, an advisory system designed mainly for crop cultivation or disease management does not necessarily address the market-related questions faced by paddy farmers, such as how marketplace procedures work, how to interpret a reference price, or how to use digital trading functionality. There is therefore a need to examine the use of retrieval-based conversational assistance within a marketplace-oriented decision-support environment.

A second research area concerns price prediction and data-driven market intelligence. Machine-learning models can estimate prices from historical observations and contextual features. However, a prediction model operating independently does not provide the complete set of functions needed to conduct a transaction. Users must still find appropriate trading partners, compare requirements, and negotiate. Furthermore, agricultural data may originate from multiple participants or geographical regions, creating potential concerns about data sharing and ownership. Federated learning offers a framework for collaborative model training without requiring all raw training data to be transferred to a central server. Yet the effectiveness of this approach depends on the training design, data distribution, model aggregation, and deployment

environment. In particular, a simulated federated-learning pipeline using synthetic data does not establish the effectiveness of a real-world, decentralized system.

A third area concerns digital agricultural marketplaces. Online platforms can improve the visibility of supply and demand by allowing users to publish listings and discover trading opportunities. However, a listing-based system may still require substantial manual effort when a user must identify which potential partners best satisfy a combination of geographical, product, price, and quantity requirements. This motivates the investigation of transparent matching methods that use multiple attributes to rank candidate partners. A weighted scoring algorithm provides an interpretable approach to this problem, although its performance must be evaluated against clearly defined criteria and should not be confused with the performance of a trained machine-learning ranking model.

A fourth area concerns automated negotiation and multi-agent systems. Intelligent agents can represent different participants and generate proposals based on role-specific information and constraints. Nevertheless, an agent's ability to generate plausible text does not guarantee that its proposed price is valid, that it respects a participant's private limit, or that the final system state remains consistent. An effective implementation must combine proposal generation with explicit validation, stopping conditions, and reliable transaction-status updates. This is particularly important in agricultural trading, where the difference between a proposed price, an accepted agreement, and a completed sale must remain clear.

These research areas demonstrate the value of the individual technologies, but they also reveal an integration challenge. Knowledge retrieval, price prediction, partner discovery, and negotiation address different stages of a trading process. If these capabilities are developed independently, the user may still need to move between separate systems or manually interpret the output of one mechanism before using another. A marketplace that coordinates these functions can provide a more continuous workflow from listing creation to partner discovery and agreement management.

For the Sri Lankan paddy supply-chain context, the individual research problem is therefore not simply to apply a language model, a price-prediction algorithm, a matching function, or negotiation agents in isolation. It is to investigate how these mechanisms can be integrated into one marketplace prototype, how their responsibilities can be clearly separated, and how the resulting workflow can be evaluated using appropriate technical evidence.

The specific research gap addressed by this dissertation is the need for an integrated and technically evaluated farmer-miller marketplace prototype that combines retrieval-grounded market assistance, a federated-learning style price reference mechanism, interpretable buyer-seller matching, and constraint validated multi-agent negotiation within a common trading workflow.

The contribution of the present study is to implement and examine this integrated approach. Its novelty should be understood in terms of the particular combination, implementation, and evaluation of the mechanisms in the intended agricultural marketplace context, rather than as a claim that each underlying technology is entirely new. The degree to which this contribution advances existing research must be established through the literature review and evaluation results presented in the subsequent chapters.

1.5 Aim and objectives

1.5.1 Research Aim

The aim of this research is to design, implement, and evaluate an AI-assisted farmer-miller digital marketplace that integrates market knowledge retrieval, reference-price prediction, weighted buyer seller matching, and multi-agent negotiation to support informed, structured, and privacy conscious paddy trading in Sri Lanka.

The research focuses on integrating these mechanisms into a functional software prototype and examining their behavior under defined test conditions. The study also considers usability, data protection, and the limitations that must be addressed before the system can be applied in a production agricultural trading environment.

1.5.2 Specific Objectives

Objective 1: Develop a digital marketplace for farmers and millers.

To implement a role-based marketplace that enables farmers to publish and manage paddy harvest listings and millers to publish and manage purchasing demands. The system should maintain relevant listing information and support the marketplace workflow required for identifying potential trading partners.

Objective 2: Implement a retrieval-grounded market advisory assistant.

To develop a RAG-based assistant that retrieves relevant information from a curated marketplace and PMB knowledge base before generating responses. The objective is to improve access to relevant information and examine the assistant's ability to provide contextual responses to supported queries in English, Sinhala, and Singlish.

Objective 3: Implement and evaluate a federated-learning based reference-price model.

To implement a simplified FedAvg training simulation and integrate the resulting linear model into the harvest-creation workflow. The objective is to generate a reference-price estimate from the supported harvest attributes and assess prediction performance using appropriate quantitative evaluation metrics. The study will explicitly identify the limitations associated with synthetic data and simulated federation.

Objective 4: Develop an interpretable buyer-seller matching algorithm.

To implement a weighted scoring mechanism that ranks compatible farmer-miller opportunities using the attributes supported by the marketplace. The objective is to make candidate discovery more structured and understandable while ensuring that listing availability and other transaction-related constraints are respected.

Objective 5: Integrate a constraint-validated multi-agent negotiation mechanism.

To develop a negotiation workflow in which farmer and miller agents generate and evaluate price proposals through sequential interactions. The objective is to combine language-model-based proposal generation with rule-based validation, private price constraints, and explicit negotiation outcomes.

Objective 6: Integrate the mechanisms into a consistent marketplace workflow.

To connect the assistant, price-reference model, matching engine, negotiation service, and transaction-management functions with the application backend, database, and user interface. The objective is to maintain consistent listing and agreement states, prevent inappropriate repeated actions, and support the expected interactions of farmer and miller accounts.

Objective 7: Evaluate the implemented prototype.

To evaluate the system using suitable unit, integration, system, regression, and non-functional tests, together with quantitative measures relevant to the individual research mechanisms. These measures may include price-prediction error, matching-score behavior, negotiation completion and validation outcomes, response time, functional correctness, and user feedback. Final claims will be based on the tests and evidence actually completed.

1.5.3 Alignment Between Objectives and Evaluation

Each objective requires a corresponding form of evaluation. Marketplace functions can be tested against their functional requirements, while the RAG assistant can be examined using a defined set of questions and answer-quality criteria. The price-prediction model requires quantitative comparison between predicted and reference values. The matching algorithm requires test cases that verify its scoring rules and ranking behavior. The negotiation system requires scenarios that examine valid proposals, private-limit enforcement, termination conditions, and agreement-status updates.

This alignment is important because the successful execution of a feature does not necessarily demonstrate that it achieves its intended research purpose. For example, producing a price prediction demonstrates that the model can generate an output, but its predictive usefulness must be examined separately. Similarly, completing a negotiation demonstrates a system outcome, but it does not by itself establish that the agreed price is economically fair. The evaluation chapter will therefore distinguish functional correctness from predictive performance, usability, and broader real-world impact.

1.6 Research questions or hypotheses

This study is guided by research questions that correspond to the identified problem, the proposed solution, and the evaluation of the implemented marketplace.

RQ1: How can a retrieval-grounded conversational assistant improve access to marketplace and PMB-related information within a farmer-miller digital marketplace?

This question examines how semantic retrieval and language-model generation can be combined to provide contextual responses to supported user queries. The evaluation will consider whether relevant information is retrieved, whether answers remain consistent with the supplied knowledge, and how the assistant handles queries expressed in different supported language styles.

RQ2: To what extent can a simplified FedAvg-based linear model generate useful reference-price estimates from the available paddy attributes?

This question examines the implementation and predictive behavior of the price-reference model. The investigation will consider the effect of the selected features, training configuration, aggregation mechanism, and dataset limitations on the resulting predictions. Quantitative performance will be reported using the evaluation metrics calculated from the actual test data.

RQ3: How effectively can a weighted rule-based algorithm rank farmer–miller opportunities according to the marketplace's trading criteria?

This question examines whether the matching algorithm applies its scoring rules consistently and ranks candidate listings in accordance with the defined compatibility factors. The evaluation will focus on scoring correctness, candidate ordering, eligibility constraints, and the interpretability of the resulting recommendations.

RQ4: How can a multi-agent negotiation mechanism combine language-model-generated proposals with rule-based constraints to support structured price bargaining?

This question investigates how the farmer and miller agents interact, how proposed actions are validated, and how the system handles acceptance, counter-offers, rejection, and termination. Particular attention will be given to enforcing the configured private price limits and recording the resulting negotiation outcome.

RQ5: To what extent can these mechanisms be integrated into a functional and consistent marketplace workflow?

This question examines the interactions between listing management, price prediction, matching, match requests, negotiation, and agreement-status management. It also considers relevant software quality attributes, including reliability, response time, usability, and access control, based on the evidence collected during testing.

These questions are exploratory and evaluation-oriented. They do not assume that the proposed mechanisms will necessarily improve farmer income, achieve market-wide price fairness, or outperform alternative systems. Such conclusions would require suitable comparative evidence and, where relevant, real-world trials.

The study does not define a single overall hypothesis that all four mechanisms will improve trading outcomes, because the mechanisms have different functions and require different evaluation methods. Instead, each research question is assessed using criteria appropriate to its technical purpose. If comparative hypotheses are introduced

later, they will be defined alongside the corresponding experimental design and supported by the collected data.

1.7 Scope limitations and assumptions

1.7.1 Research Scope

The scope of this dissertation is limited to the individual AI-powered farmer-miller marketplace component of the wider Digital Goviya research ecosystem. It covers the design, implementation, integration, and technical evaluation of the marketplace functions required to support paddy trading between farmers and millers.

The application scope includes farmer and miller accounts, harvest and demand listing management, partner connections, matching recommendations, match requests, negotiation workflows, and agreement-related status management. The implementation also includes an advisory assistant, a reference-price prediction mechanism, and a weighted matching algorithm.

The RAG assistant is limited to the curated knowledge available to it, including supported marketplace functionality and PMB-related information. It is not intended to replace an official government information service or provide unrestricted agricultural advice. Its responses depend on the content and quality of the available knowledge base.

The price-prediction scope covers the implemented simplified FedAvg simulation and the associated linear prediction model. The current prototype uses synthetic training data and simulated clients. The research therefore focuses on demonstrating the training and inference workflow and evaluating the model under the available experimental conditions, rather than claiming deployment across real farms, mills, or independent devices.

The matching scope covers the implemented rule-based scoring mechanism and its supported listing attributes. The current scoring scheme uses district, paddy type, price closeness, and quantity suitability. The matching algorithm does not constitute a trained machine-learning ranking model, and its score should be interpreted as an indicator of compatibility according to the selected rules rather than a guaranteed probability of successful trading.

The negotiation scope covers sequential interaction between a farmer agent and a miller agent, using language-model-generated proposals and deterministic validation rules. The study examines how the system handles offers, counter-offers, acceptance, rejection, and termination under the configured constraints. It does not claim that automated negotiation always identifies the economically optimal price or guarantees a mutually beneficial outcome.

The implementation uses a mobile-oriented interface developed with Expo and React Native, a Node.js and Express backend, MongoDB for application data, and Python services for selected AI functions. OpenAI is used for RAG response generation, while the current negotiation implementation uses a OpenAI model. The technical evaluation focuses on the functions and services included in the individual component.

1.7.2 Limitations

Synthetic training data. The current price-prediction prototype uses synthetic data generated for experimentation rather than a comprehensive, verified dataset of actual farmer–miller transactions. Consequently, the prediction results may not reflect real market conditions, seasonal fluctuations, policy changes, or regional price differences.

Simulated federated learning. The implemented FedAvg process represents multiple clients within a single execution environment. It demonstrates model training and parameter aggregation concepts but does not establish that the system has been deployed across independent devices or organizations. It also does not, by itself, guarantee privacy against model-inference attacks or other threats.

Limited knowledge-base coverage. The RAG assistant retrieves information from a curated knowledge base. Its coverage is limited to the information supplied to the system, and its answers may be incomplete when the required information is missing or cannot be retrieved. The assistant should not be regarded as a live, comprehensive search service for all government publications.

Rule-based matching. The matching score depends on the selected attributes, weights, and threshold rules. It does not currently incorporate every factor that may affect real trading decisions, such as comprehensive grain-quality measurements, verified transportation costs, dynamic mill capacity, or a validated model of buyer behavior. The score therefore indicates compatibility within the defined implementation rather than overall economic suitability.

Negotiation-model limitations. The negotiation system relies on a language model and a configured set of validation rules. Its behavior may vary according to model responses, prompts, and negotiation conditions. Rule-based validation can constrain supported decisions, but the overall mechanism requires further testing before it can be relied upon for high-value real-world transactions.

Privacy and security limitations. The application includes authentication, role-based access controls, and measures intended to prevent unnecessary exposure of private negotiation limits. Nevertheless, application-level separation should not be equated with cryptographic confidentiality. Persistent storage, API responses, logging, access permissions, and deployment configuration must all be evaluated before making stronger privacy claims.

Limited real-world evaluation. The prototype's technical results and any feedback collected during the study may not represent the experiences of the entire Sri Lankan farming and milling community. The extent of generalization will depend on the participant sample, test scenarios, data quality, and evaluation procedures actually completed.

Integration dependencies. The marketplace depends on the correct operation of its frontend, backend, database, and supporting Python services. Service availability, network conditions, model execution time, and configuration can influence the overall user experience.

These limitations define the boundaries of the research and identify opportunities for future development. They do not remove the value of implementing and evaluating a prototype, but they must be considered when interpreting its results.

1.7.3 Research Assumptions

The research makes several assumptions to support implementation and evaluation.

First, farmers and millers are assumed to provide reasonably accurate information when creating harvest and demand listings. Incorrect quantities, prices, or paddy-type selections can affect the relevance of matching recommendations and negotiation outcomes.

Second, the selected listing attributes are assumed to be sufficient for demonstrating the basic matching workflow. The resulting scores are not assumed to represent every commercial factor that influences a real trading decision.

Third, the synthetic dataset is assumed to be suitable for demonstrating the operation of the price-prediction and simulated aggregation pipeline. It is not assumed to represent the full distribution of actual Sri Lankan paddy prices.

Fourth, the curated knowledge base is assumed to contain relevant information for the categories of questions the RAG assistant is designed to answer. The study does not assume that every possible user question can be answered correctly.

Fifth, the language model is assumed to generate proposals and responses that can be processed by the application's validation and response-handling mechanisms. This assumption is tested through defined scenarios and does not imply that all generated outputs will be valid without validation.

Finally, the marketplace is assumed to operate in an environment where users can access the application and supporting services. Wider adoption, market participation, and integration with external agricultural institutions are outside the scope of the current prototype unless explicitly demonstrated by the evaluation.

1.8 Individual contribution

This dissertation focuses on the AI-Powered Farmer–Miller Marketplace, identified as C03 within the collaborative Digital Goviya research project. The overall project is developed through the contributions of four related research components. The individual contribution reported in this dissertation is restricted to the design, implementation, integration, and evaluation of the C03 marketplace and its associated mechanisms.

The principal contribution is the development of a marketplace workflow that connects digital harvest and demand listings with price-reference prediction, candidate matching, and assisted negotiation. The implementation combines mobile application screens, backend APIs, persistent storage, and supporting AI services so that the individual features can operate within a common application.

The first contribution is the implementation of a retrieval-grounded conversational assistant. This feature uses a curated knowledge base, multilingual embeddings, FAISS similarity search, and an OpenAI language model to generate contextual responses to supported marketplace and PMB-related questions. The implementation also includes query processing and the preparation of relevant user context, subject to privacy-related filtering.

The second contribution is the implementation of a simplified federated-learning-style price-prediction pipeline. This includes preparing the supported features, training a linear model through simulated clients, aggregating model parameters using a FedAvg-style process, saving the resulting model artefacts, and integrating inference into the harvest workflow. The contribution demonstrates an end-to-end technical

pipeline while explicitly recognizing that the current training data is synthetic and the client environment is simulated.

The third contribution is the development of an interpretable weighted matching algorithm. The algorithm evaluates the supported compatibility factors and produces a score that can be used to rank potential farmer-miller opportunities. The implementation also connects matching recommendations with listing eligibility and match-request workflows.

The fourth contribution is the implementation of a multi-agent negotiation workflow. This includes the farmer and miller agent logic, the negotiation orchestration service, integration with the locally hosted language model, rule-based decision validation, and the recording of negotiation outcomes. The purpose of the validation layer is to enforce configured negotiation constraints and maintain a structured process rather than relying solely on unconstrained language-model responses.

In addition to these research mechanisms, the individual component includes supporting marketplace functions such as authentication, role-specific interfaces, listing management, partner connections, match requests, notifications, and agreement-related status handling. These functions are important because they enable the core research mechanisms to be exercised through a complete application workflow.

The contribution also includes the integration of the frontend, backend, database, and supporting services, as well as the preparation and execution of appropriate tests. The evaluation should identify which requirements have been verified, which performance measures have been calculated, and which aspects require further investigation.

The individual contribution must be distinguished from the contributions of the other three research components. The C03 dissertation does not claim sole authorship of the overall Digital Goviya ecosystem, the digital twin component, the separate price-forecasting component, or the blockchain-based warehouse component. Any

interfaces or data exchanges involving these components should be described according to the integration that has actually been implemented and verified.

Similarly, the research does not claim the use of production-grade federated-learning infrastructure, a trained machine-learning matching model, or cryptographic protection of negotiation limits unless such capabilities are implemented and independently verified. This distinction ensures that the reported contribution accurately reflects the current software artefact and its demonstrated behavior.

Overall, the individual contribution lies in bringing market knowledge access, reference-price estimation, interpretable matching, and constraint-validated negotiation together in a functional farmer–miller marketplace prototype and examining its technical capabilities and limitations in the context of Sri Lanka's paddy supply chain.

1.9 Thesis structure

This dissertation is organized into chapters that progressively explain the research context, the design and implementation of the proposed marketplace, and the evaluation of the resulting prototype.

Chapter 1: Introduction presents the overall Digital Goviya research context and introduces the individual AI-powered farmer-miller marketplace. It defines the research problem, identifies the research gap, states the research aim and objectives, formulates the research questions, establishes the scope and limitations, and explains the individual contribution.

Chapter 2: Literature Review examines relevant research on digital agricultural marketplaces, retrieval-augmented generation, multilingual information retrieval, federated learning, price prediction, weighted matching, and multi-agent negotiation. It compares existing approaches, identifies their limitations, and establishes the theoretical and technical foundation for the proposed marketplace.

Chapter 3: Research Methodology and System Design explain the research approach, requirements analysis, system architecture, data flow, technology selection, and design of the major marketplace mechanisms. It describes the methodological decisions used to implement the advisory assistant, simulated federated-learning pipeline, matching algorithm, and negotiation system. It also discusses relevant ethical, privacy, and commercialization considerations.

Chapter 4: System Implementation presents the implementation of the marketplace frontend, backend, database interactions, authentication, and supporting AI services. It explains the integration of the major features, the principal algorithms, and the application workflows required to support farmer and miller interactions.

Chapter 5: Testing, Results and Discussion presents the testing strategy, experimental setup, evaluation metrics, and results obtained from the implemented system. It discusses functional correctness, price-prediction performance, matching behavior, negotiation outcomes, relevant non-functional requirements, and user feedback where available. It also examines the limitations of the results and their implications for the research questions.

Chapter 6: Conclusion and Recommendations summarize the research, reviews the extent to which the objectives have been achieved, and identifies the principal contributions and limitations. It also presents recommendations for future work, including evaluation with real agricultural datasets, more realistic federated-learning deployments, enhanced matching criteria, stronger privacy controls, and broader field testing.

The dissertation concludes with the reference list and appendices containing supplementary material, such as detailed test cases, additional results, questionnaires, system screenshots, and supporting implementation evidence where appropriate.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

2.1.1 Purpose of the Literature Review

This chapter reviews the concepts and research relevant to an AI-powered farmer–miller marketplace for Sri Lanka's paddy supply chain. It focuses on digital agricultural marketplaces, market information and price discovery, retrieval-augmented generation (RAG), federated learning (FL), buyer–seller matching, and multi-agent negotiation. These areas provide the conceptual basis for a system that brings together listing management, market-information assistance, price-reference support, trading-partner discovery, and structured negotiation.

The review has two purposes: to establish the rationale for the selected techniques and to identify how they can be combined into a coherent decision-support workflow. It does not claim that the proposed marketplace replaces existing agricultural institutions or independently resolves all supply-chain challenges. Rather, it examines how established computing approaches can be adapted to support selected activities in the Sri Lankan paddy-trading context.

2.1.2 Review approach and source selection

The chapter uses a focused narrative review of research and technical literature on digital agriculture, RAG , federated learning, recommendation and matching, and multi-agent systems. Sources were selected for their relevance to the system's main functions and for their ability to explain the purpose, operation, and design considerations of the techniques used. Foundational publications were prioritised for core methods, including the smart-farming review by Wolfert et al. (2017), the RAG framework described by Lewis et al. (2020), the federated-learning survey by Kairouz et al. (2021), the FedAvg approach by McMahan et al. (2017), and the multi-agent systems text by Wooldridge (2009).

The review is organised thematically rather than as a list of papers. It first establishes the domain and theoretical foundations, then discusses related research and categories of existing systems, and finally synthesises the research gaps into a conceptual

framework. The chapter therefore connects each technical area to a marketplace requirement and to the evidence that would be appropriate for evaluating the implemented functions.

2.1.3 Scope and Boundaries

The review concentrates on the AI-powered farmer–miller marketplace and the technologies directly supporting its workflow. Crop disease detection, irrigation optimization, agricultural robotics, and the detailed implementation of other Digital Goviya components are outside this chapter's scope except where they provide context for the marketplace. Blockchain and warehouse coordination are also not reviewed as marketplace functions in their own right.

The discussion distinguishes between established findings in published research, the design rationale for the marketplace, and outcomes that require evaluation. In particular, the literature on federated learning is used to explain distributed training principles, not to imply that every federated-learning deployment automatically provides privacy. Likewise, RAG is considered a way to connect language generation with retrieved knowledge, rather than a guarantee that every generated answer is correct. These distinctions help align the literature review with the actual purpose and evidence of the prototype.

2.2 Domain and theoretical foundations

2.2.1 Sri Lankan Paddy Supply Chain and Digital Marketplaces

The paddy supply chain connects farmers who produce and sell harvested paddy with millers who procure it for processing. Trading decisions can involve the quantity available, paddy variety, location, offered price, purchasing requirements, and the timing of a potential transaction. When this information is distributed across different participants and channels, a farmer or miller may need to compare several sources before identifying a suitable trading opportunity.

Digital marketplaces provide a structured environment for publishing supply and demand information and discovering potential trading partners. Their value depends

not only on listing availability but also on the quality and usability of the information presented. Matching functions can organise candidate listings, while market-information tools can help users interpret relevant information before contacting another participant. For the Sri Lankan context, the marketplace is designed around farmer harvest listings and miller demand listings, with the intention of supporting clearer discovery and decision-making rather than replacing participants' own judgement.

Wolfert et al. (2017) discuss how data and digital technologies are becoming important to agricultural decision-making and farm management. Their review provides a broader basis for considering digital services as part of an agricultural information environment. A trading marketplace applies this general principle to a specific part of the supply chain: bringing together records and decision-support functions that are relevant to the exchange between farmers and millers.

2.2.2 Information Asymmetry and Price Discovery

Information asymmetry occurs when participants in a transaction have different access to relevant information. In agricultural trading, parties may differ in their knowledge of current offers, demand conditions, purchasing procedures, or prices in other locations. This can make it more difficult to assess whether a proposed transaction is suitable. A digital system can help organise available information, but its effectiveness depends on the accuracy, coverage, and timeliness of the records it uses.

Price discovery is the process through which participants form an understanding of a plausible transaction price using available market information and negotiation. A price estimate can be useful as a reference point, but it should not be treated as a guaranteed price or as a replacement for market judgement. The final transaction may also depend on factors not represented in a model, including quality, transport arrangements, timing, and the preferences of the parties.

The marketplace therefore treats price-reference support as one input to a broader decision process. A reference value can be considered alongside listing attributes, matching results, available information, and the offers exchanged during negotiation.

This framing keeps the role of prediction clear: it informs decisions rather than determines the outcome of a transaction.

2.2.3 Digital Marketplaces as Decision-Support Systems

A decision-support system combines data, analytical methods, and user-facing functions to assist decisions that may involve several alternatives. In the marketplace context, the user needs to find relevant listings, compare attributes, interpret supporting information, and decide whether to negotiate. Bringing these activities into one workflow can reduce the need to treat each activity as an isolated task.

The functions in the farmer–miller marketplace have complementary roles. Listing management represents supply and demand; matching ranks potential partners according to stated compatibility criteria; price-reference support provides an additional point of comparison; RAG provides responses grounded in a curated knowledge collection; and negotiation agents help structure the exchange of offers. The design combines these functions while retaining the user's role in deciding whether to proceed with a trading opportunity.

The system's overall usefulness must be examined at both module and workflow levels. A module can operate as intended in isolation, while integration testing is needed to establish that information is passed correctly between services and that marketplace states are updated consistently. This distinction provides a basis for the evaluation approach described in later chapters.

2.2.4 Retrieval-Augmented Generation for Knowledge Assistance

Large language models can produce fluent natural-language responses, but a response generated without relevant source context may not reflect the specific information a user needs. Retrieval-augmented generation addresses this by retrieving relevant information and using it as context for generation. Lewis et al. (2020) present RAG as a framework that combines parametric language-model knowledge with an external, retrievable non-parametric memory.

A typical RAG workflow has two stages. During preparation, documents or knowledge records are converted into representations suitable for retrieval and indexed with associated metadata. At query time, a user's question is transformed into a query

representation, relevant records are retrieved, and the selected context is supplied to the language model to support answer generation. Retrieval quality and source coverage are important because generation is constrained by the relevance and completeness of the available context.

For the farmer–miller marketplace, this approach supports a question-answering assistant that uses curated marketplace and PMB-related knowledge records. Multilingual embeddings are intended to improve retrieval for questions expressed in different language forms. The assistant's role is to provide contextual information, while users should still be able to consult authoritative sources for consequential decisions. The design is therefore knowledge-grounded assistance rather than an unrestricted claim of factual verification.

2.2.5 Federated Learning and Privacy-Aware Price Prediction

Federated learning is a distributed machine-learning approach in which participating clients train locally and contribute model updates to a shared model, reducing the need to transfer raw training data to a central location. McMahan et al. (2017) introduced Federated Averaging (FedAvg), which aggregates locally computed model updates to form a global model. Kairouz et al. (2021) review the broader research landscape and discuss the practical challenges and open questions associated with federated learning.

The general training cycle involves distributing a current global model, performing local training, collecting model updates, aggregating those updates, and repeating the process. The approach can be relevant where data is held by separate participants or organizations. However, federated learning is not, by itself, a complete privacy guarantee. Model updates can still require security and privacy analysis, and the design must specify what information is shared and how it is protected.

In the marketplace, federated learning provides the conceptual basis for a price-reference model organized around simulated district clients. The model is intended to produce an indicative reference that can be considered during trading decisions. The literature supports the choice of a federated training structure as a research technique; claims about predictive usefulness must be based on the model's evaluation, and claims about privacy must match the protections actually implemented.

2.2.6 Buyer-Seller Matching and Recommendation

Matching and recommendation techniques help users identify relevant options from a set of candidates. A marketplace may use filters to remove unsuitable listings and ranking criteria to order the remaining candidates. The choice of criteria depends on the application domain and should reflect the attributes that are available and meaningful for the decision.

Rule-based weighted matching assigns scores to defined criteria and combines them into an overall ranking. Its main design advantage is interpretability: each score can be traced to a stated criterion. The approach is suitable when the requirements are explicit and a transparent ranking process is preferred. The ranking quality nevertheless depends on the appropriateness of the selected criteria and weights, so test cases should verify both score calculation and candidate filtering.

The farmer–miller marketplace uses district, paddy type, price closeness, and quantity compatibility as its specified matching criteria. The result is a ranked set of potential trading opportunities rather than a guarantee that a transaction will be successful. This makes the matching process explainable and provides a clear basis for testing expected results against known listing combinations.

2.2.7 Multi-Agent Systems and Automated Negotiation

A multi-agent system contains multiple agents that act within an environment and may have different goals, information, or constraints. Wooldridge (2009) describes multi-agent systems as a field concerned with autonomous agents and their interactions. Negotiation is one relevant interaction pattern: participants exchange proposals and responses in an attempt to reach an outcome acceptable under their respective conditions.

In a digital marketplace, a farmer and a miller may value a proposed price differently. A negotiation workflow can represent these perspectives through separate farmer and miller agents, each guided by a role-specific prompt and applicable constraints. An orchestration service manages the sequence of actions, maintains negotiation state and history, and checks whether the process should continue or terminate.

Language models can generate flexible offers and counteroffers, while a deterministic validator can check whether generated actions comply with defined rules. This combination separates natural-language proposal generation from rule enforcement. The validator does not make every negotiation outcome optimal, but it provides a clear control point for enforcing constraints and supporting repeatable tests. In the implemented design, the two agents are orchestrated sequentially, and the final negotiation result remains distinct from the completion or fulfilment of a marketplace transaction.

2.2.8 Integration of the Theoretical Foundations

The reviewed theories support different parts of the trading workflow. Digital marketplace principles organise supply and demand information. RAG connects natural-language responses to a retrievable knowledge source. Federated learning provides a model-training structure based on local client updates and aggregation. Weighted matching ranks candidate listings according to explicit criteria. Multi-agent negotiation represents the exchange of proposals between parties with different roles and constraints.

These techniques are complementary rather than interchangeable. RAG addresses knowledge access; price-reference prediction estimates a value; matching compares listings; and negotiation structures an interaction. Their integration offers a coherent decision-support process, while each module still requires an evaluation method suited to its purpose. The conceptual framework in Section 2.5 brings these relationships together.

2.3 Related research

2.3.1 Digital Technologies in Agriculture and Supply Chains

Research on digital agriculture examines the use of data, connectivity, analytics, and information systems to support agricultural activities. Wolfert et al. (2017) review the role of big data in smart farming and describe the increasing significance of data-

driven approaches in agricultural management. This work provides context for digital services that organise information and support decisions across agricultural processes.

The farmer–miller marketplace applies this broad direction to trading coordination. Rather than attempting to cover every stage of agricultural production, it focuses on listing information, partner discovery, price-reference support, knowledge assistance, and negotiation. This narrower scope makes it possible to evaluate the functions and their interactions within a defined marketplace workflow.

2.3.2 Retrieval-Augmented Generation and Knowledge-Intensive Assistance

Lewis et al. (2020) introduce RAG as an approach that combines information retrieval with text generation. The key relevance to a marketplace assistant is the ability to provide a language model with context retrieved from a defined knowledge collection. Instead of relying only on the model's general learned parameters, the response process can draw on records selected for the user's question.

The practical performance of a RAG application depends on the content of its knowledge base, the embedding and retrieval methods, and the way the retrieved material is incorporated into the prompt. Consequently, an evaluation should examine not only whether an answer is produced but also whether relevant records were retrieved and whether the response reflects those records. The marketplace assistant follows this design principle by separating knowledge preparation, retrieval, and answer generation.

2.3.3 Federated Learning for Distributed Model Training

McMahan et al. (2017) present FedAvg as a method for aggregating locally trained model updates into a shared model. Kairouz et al. (2021) survey subsequent federated-learning research, including issues related to statistical and system heterogeneity, communication, security, and privacy. Together, these publications establish the conceptual foundation for collaborative training without requiring all raw training records to be pooled centrally.

For a marketplace price-reference function, a client-based training structure offers a way to represent distributed data ownership. The implementation uses a simplified simulation to explore the local-training and aggregation workflow. The relevant

research connection is the federated-learning process itself; the suitability of the resulting price model must be assessed separately using the evaluation evidence reported in the results chapter.

2.3.4 Research on Intelligent Matching and Recommendation

Matching systems range from simple filtering to weighted ranking and learned recommendation models. Filtering determines which candidates meet required conditions, while ranking orders candidates according to selected attributes. For marketplaces with explicit business rules, a weighted approach offers traceability because the contribution of each criterion can be inspected.

The selected matching approach ranks harvest and demand listings using district, paddy type, price closeness, and quantity compatibility. This design supports transparent test cases, including exact matches, partial compatibility, and unavailable listings. It is intended to help users prioritise candidates; the user remains responsible for deciding whether a particular opportunity is appropriate.

2.3.5 Automated Negotiation and Multi-Agent Systems

Wooldridge (2009) provides foundational coverage of agents and multi-agent systems, including interactions among agents operating with different objectives. The idea of separate role-based agents is relevant to a farmer–miller negotiation because each participant may have distinct acceptable conditions. A structured negotiation process can manage proposals, counteroffers, state transitions, and termination conditions.

The marketplace adapts this concept using a farmer agent and a miller agent within an orchestrated service. Language-model-generated actions are checked by a rule-based validator before the negotiation state is updated. This arrangement is designed to combine flexible proposal generation with explicit constraints. Its effectiveness can be assessed through tests of valid actions, state transitions, and termination behavior.

2.3.6 Synthesis of Related Research

The reviewed literature provides established foundations for the marketplace's principal techniques, but each publication addresses a particular method or research area. Wolfert et al. (2017) provide the wider digital-agriculture context; Lewis et al.

(2020) explain retrieval-augmented generation; McMahan et al. (2017) and Kairouz et al. (2021) inform the federated-learning design; and Wooldridge (2009) provides the agent-based foundation for modelling negotiation.

The marketplace's research focus is the integration of these techniques with listing management and farmer–miller trading activities. Its contribution is therefore best described in terms of the design and evaluation of a combined prototype, rather than as the invention of RAG, federated learning, matching, or multi-agent systems themselves.

2.4 Existing systems products and techniques

2.4.1 Categories of Existing Agricultural Systems

Agricultural digital systems differ in the problems they address. Information services provide market prices, purchasing procedures, or agricultural guidance. Digital trading platforms support the publication and discovery of supply and demand. Predictive systems estimate values such as prices or yields, while recommendation systems rank available options. Knowledge assistants enable users to ask questions in natural language, and negotiation tools support exchanges of offers and counteroffers.

These categories may overlap, but they do not necessarily provide the same capabilities. A platform that publishes market information does not automatically support partner matching, and a prediction service does not necessarily manage a trading workflow. For this reason, comparison should focus on capabilities and evidence rather than assuming that all agricultural systems are direct substitutes for the proposed marketplace.

2.4.2 Comparison of Marketplace Approaches

Manual search and filtering allow users to inspect listings directly, but users may need to compare many records themselves. Rule-based matching adds a ranking layer using explicit criteria. Machine-learning recommendation can learn patterns from data, while its suitability depends on the availability and representativeness of training examples.

The marketplace uses rule-based weighted matching because its current criteria are explicit and can be tested against expected scores.

Similarly, general language-model assistance and RAG differ in how they use external information. RAG retrieves relevant records before generating a response, making the source context part of the response workflow. Rule-based negotiation and language-model-assisted negotiation also serve different purposes: rules offer predictable outcomes for predefined cases, while language models can generate more flexible proposals. Combining model-generated actions with a validator provides a way to retain explicit controls in the negotiation process.

2.4.3 Comparison of Technical Approaches

The technical approaches relevant to marketplace differ in their data requirements, interpretability, and expected role within the marketplace.

Approach	Primary purpose	Main advantage	Main limitation
Manual search and filtering	Find suitable listings	Simple and user-controlled	May require repeated manual comparison
Rule-based weighted matching	Rank potential trading partners	Transparent and relatively easy to inspect	Depends on selected criteria and weights
Centralized machine learning	Learn patterns from combined training data	Centralized training and evaluation can be straightforward	Requires data consolidation and suitable access controls
Federated learning	Train a shared model from distributed client updates	Can reduce the need to centralize raw training data	Requires appropriate client separation, aggregation and privacy assessment
Conventional language-model assistance	Generate answers to user questions	Flexible natural-language interaction	May generate unsupported or outdated information
Retrieval-augmented generation	Generate answers using retrieved source context	Connects responses to an external knowledge collection	Depends on source coverage and retrieval quality

Rule-based negotiation	Apply predefined offer and acceptance rules	Predictable and easier to validate	Limited adaptability to complex situations
LLM-assisted negotiation with validation	Generate proposals and check them against rules	Combines flexible proposal generation with explicit constraints	Depends on model consistency, validation rules and negotiation design

Table 1 : Comparison of relevant technical approaches

The comparison indicates that no single approach addresses every requirement of the marketplace. Rule-based matching offers transparency but requires careful criterion selection. Federated learning offers a distributed training framework but does not automatically guarantee privacy. RAG connects language generation to an external knowledge source but depends on retrieval quality. LLM-assisted negotiation can generate flexible proposals but needs deterministic constraints to reduce invalid decisions.

Marketplace uses a combination of these approaches to address distinct tasks. The choice is appropriate for a prototype intended to demonstrate an integrated workflow, provided that the limitations are acknowledged and each module is evaluated using suitable criteria.

2.4.4 Existing Systems and Evidence Requirements

A fair comparison of existing agricultural systems requires consistent criteria. Relevant questions include which user groups are supported, whether supply and demand listings can be managed, what market information is available, whether matching or price prediction is provided, how knowledge questions are answered, whether negotiation is supported, and what evidence is published about performance and privacy. The following framework can be used when assessing specific products or systems for which reliable documentation is available.

Comparison criterion	Questions to investigate
User groups	Does the system support farmers, millers, or other agricultural participants?
Listing management	Can users publish and manage supply and demand records?
Market information	Does it provide prices, purchasing procedures, or other market information?
Matching	Does it automatically rank or recommend trading partners?
Price prediction	Does it provide a predictive estimate, and what data is used?
Knowledge assistance	Does it answer natural-language questions using a defined knowledge source?
Negotiation	Does it support offers, counteroffers, or automated negotiation?
Privacy	What information is collected, stored, shared, or processed locally?
Evaluation evidence	Are performance measures or user-study results published?
Local relevance	Does it support the requirements and context of Sri Lankan paddy trading?

Table 2 : Recommended framework for comparing existing systems

2.4.5 Rationale for the Selected Techniques

The selected techniques were chosen to align with different marketplace tasks. Rule-based weighted matching provides an inspectable ranking process using listing attributes. RAG supports natural-language access to a curated collection of marketplace and PMB-related knowledge. A federated-learning-style workflow

provides a basis for exploring collaborative model training through simulated district clients. The negotiation service uses separate farmer and miller agents, with a rule-based validator checking generated actions against the defined constraints.

The OpenAI API is used for language-model inference in the updated negotiation implementation; the negotiation service orchestrates the agents and manages the workflow. This distinction matters because the model supplies generated proposals, while application-level rules determine whether an action is accepted by the workflow. The selected techniques are integrated as decision-support functions, and their performance is assessed according to their individual roles rather than through a single undifferentiated measure.

2.5 Gap synthesis and conceptual framework

2.5.1 Synthesis of the Identified Gaps

The literature establishes strong foundations for digital agricultural information services, knowledge-grounded language generation, distributed model training, matching, and agent-based interaction. The design opportunity addressed by the farmer–miller marketplace is to bring these capabilities into a connected workflow tailored to paddy trading. The following synthesis identifies the design needs that follow from the reviewed work.

Gap 1: Decision support across the trading workflow. Listing management, market information, partner discovery, price references, and negotiation are distinct functions. A connected marketplace workflow can make these functions available within one environment and allow their interactions to be evaluated through end-to-end scenarios.

Gap 2: Contextual knowledge assistance. Natural-language assistance is more useful when its responses are supported by relevant domain information. RAG provides a retrieval-and-generation pattern for grounding responses in a curated knowledge collection. Evaluation should therefore consider retrieval relevance as well as response quality.

Gap 3: Price-reference support. A model-generated price value needs to be presented as an estimate that supports, rather than replaces, user judgement. A clearly labelled reference output and suitable predictive evaluation can help users interpret the role of the model within the trading process.

Gap 4: Explainable trading-partner discovery. Matching results should reflect stated compatibility criteria. Weighted scoring provides a transparent way to compare candidate harvest and demand listings, and controlled test cases can verify the calculation and ranking rules.

Gap 5: Structured negotiation. Farmer and miller participants can have different goals and constraints. A two-agent workflow can model these roles, while validation rules and state management help keep the exchange within defined conditions and support predictable termination.

Gap 6: Accurate representation of privacy-related design. Federated learning can reduce the need to centralise raw training records, but the level of privacy protection depends on the full implementation. The framework therefore distinguishes a federated-learning training structure from additional protections such as secure aggregation or differential privacy, which should only be claimed when implemented and evaluated.

Gap 7: Evidence at both module and integration levels. Individual functions require appropriate tests, and an integrated workflow requires tests of service communication, state transitions, and error handling. The evaluation should link each design objective to evidence suited to the corresponding function.

2.5.2 Proposed Conceptual Framework

The conceptual framework represents how marketplace information and AI-assisted functions contribute to trading decisions. It connects the system's inputs, processing modules, outputs, and intended outcomes.

At the input level, the marketplace receives farmer harvest listings and miller demand listings. These records may include location, paddy type, quantity, and price-related attributes. The knowledge-assistance module uses a curated collection of marketplace and PMB-related information. The price-reference module uses the features and data

available to its model, while negotiation uses the relevant trading context and configured participant constraints.

The processing level contains four main functions: RAG-based knowledge retrieval and response generation; price-reference prediction using a federated-learning-style workflow; rule-based weighted matching; and farmer–miller agent negotiation. Each function transforms its inputs into an output that supports a particular decision. Their integration allows the marketplace to present contextual information, an indicative price reference, ranked trading opportunities, and negotiation outcomes.

The intended outcomes are improved access to relevant information, more structured partner discovery, an additional reference for price decisions, and a defined negotiation workflow. These are design objectives rather than assumed results. Their value is assessed through module-specific tests and integrated workflow evaluation.

2.5.3 Mapping the Gaps to the C03 Design

Identified gap	**C03 design response**	**Evidence required**
Decision support is distributed across different activities	Integrates listings, matching, price references, assistance, and negotiation	End-to-end workflow tests
Users need contextual access to marketplace knowledge	RAG assistant with curated knowledge and multilingual query handling	Retrieval relevance, answer correctness, and unsupported-answer tests
Price estimates can be misunderstood as guaranteed prices	Presents the model output as a reference estimate	Prediction metrics, baseline comparison, and clear output labelling
Manual search may require repeated comparisons	Weighted ranking of harvest–demand candidates	Test cases for compatibility, ranking, and exclusion rules

Negotiation can be inconsistent or difficult to validate	Farmer and miller agents with deterministic validation	Constraint-compliance tests, agreement rate, and round counts
Privacy claims require technical evidence	Simplified FL-style training simulation	Documented client setup, aggregation tests, and explicit privacy limitations
Individual AI modules may fail when integrated	Defined service interactions and marketplace state transitions	Integration tests and error-handling tests

Table 3 : Mapping identified gaps to Marketplace features and evaluation

The table links each identified gap to a design response and a form of evidence that could establish whether the response is effective. This is important because implementing a feature is not the same as demonstrating that it solves the associated problem. The evaluation must show whether the implementation behaves as intended and identify any remaining limitations.

2.5.4 Conceptual Relationships and Research Propositions

The framework suggests several relationships for evaluation. First, the relevance of retrieved knowledge is expected to influence the relevance and supportability of RAG responses. Second, the usefulness of matching results depends on listing-data quality and the suitability of the scoring criteria. Third, price-reference performance depends on the relationship between the model's input data and the target market conditions. Fourth, negotiation behavior depends on the agents' prompts, the validator's rules, and correct state management. Finally, the integrated workflow depends on consistent data exchange and state transitions between the marketplace services.

These are evaluation propositions, not confirmed findings. They guide the choice of test cases and measures in the methodology and results chapters. The evidence should be reported according to what was actually tested, with intended benefits distinguished from measured outcomes.

2.5.5 Summary of the Research Contribution

The literature review establishes the theoretical basis for combining digital marketplace functions with RAG-based knowledge assistance, federated-learning-style price-reference modelling, rule-based weighted matching, and multi-agent negotiation. Each technique addresses a different requirement: knowledge access, indicative price support, partner discovery, or structured exchange of offers.

The research contribution is the design, implementation, and evaluation of these functions within an AI-powered farmer–miller marketplace tailored to the Sri Lankan paddy supply-chain context. The conceptual framework connects the techniques to their inputs, outputs, intended decision-support outcomes, and evaluation criteria. The next chapter describes the methodology used to implement and assess the marketplace, including the architecture, module-level testing, integration testing, and evaluation procedures.

3 RESEARCH METHODOLOGY

3.1 Research design

This research follows a design and development approach to implement and evaluate an AI-powered farmer-miller marketplace for the Sri Lankan paddy supply chain. The research focuses on developing a functional software prototype that integrates marketplace operations with AI-assisted information retrieval, price-reference prediction, buyer-seller matching, and automated negotiation.

A prototype-based research approach is appropriate because the main objective is to design, implement, and evaluate a technical solution to the identified marketplace coordination problem. The research combines software development, controlled testing, quantitative performance evaluation, and qualitative analysis of system behavior.

The individual component, marketplace, is developed as part of the wider Digital Goviya ecosystem. Its evaluation concentrates on the functions implemented within the marketplace component rather than attempting to evaluate every component of the overall research project.

The research follows four main activities:

1. **Requirement identification:** identifying the marketplace requirements and the expected behavior of each feature from the PMB regional manager.
2. **System design and implementation:** developing the frontend, backend services, data management, and AI-assisted modules.
3. **Testing and evaluation:** examining functional correctness, model performance, matching behavior, negotiation constraints, and integration.
4. **Analysis and reporting:** interpreting the observed results, identifying limitations, and documenting possible improvements.

The research does not assume that implementing AI-based features automatically improves farmer income, guarantees fair prices, or provides complete privacy

protection. These outcomes require evidence beyond the successful operation of the prototype.

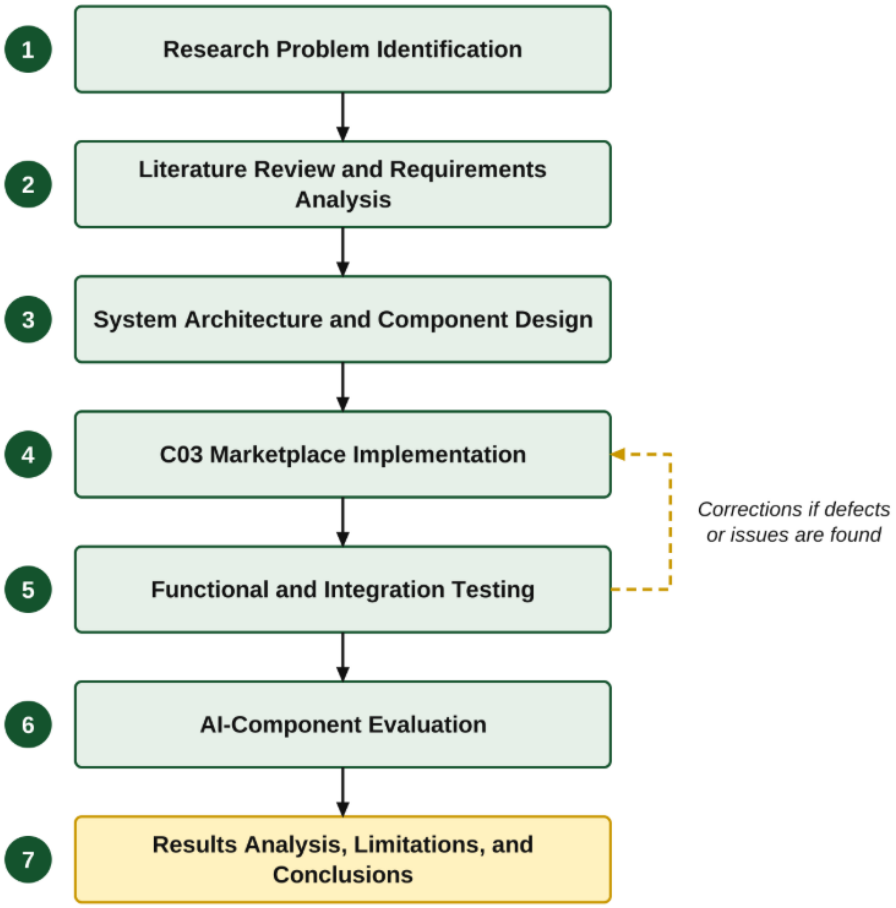


Figure 2 : Research and Development Methodology

3.2 Data participants or study materials

The study uses software components, structured data, a curated knowledge collection, and controlled test scenarios as its primary study materials. The prototype is evaluated through technical testing rather than through a confirmed large-scale field study involving real farmers and millers.

3.2.1 Marketplace Data

The marketplace stores information relating to farmer harvest listings, miller demand listings, user accounts, matching records, notifications, and negotiation activities.

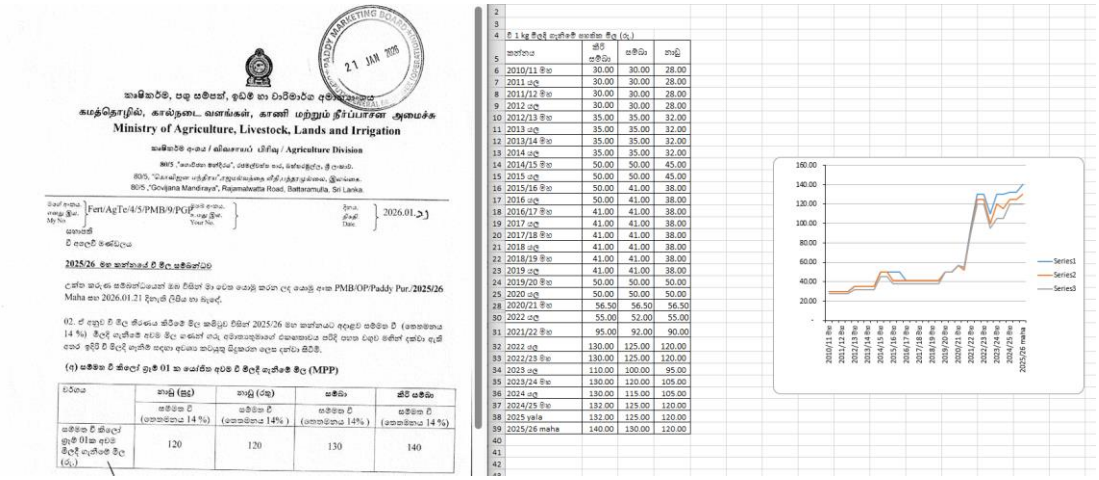
Relevant listing attributes include district, paddy type, available or required quantity, and price-related information.

These records support the main marketplace workflow, including listing management, matching, and negotiation. The system uses application-level authentication and role-based behavior to distinguish farmer and miller activities.

Any evaluation involving real personal or commercial records must be conducted using appropriate authorization and data-protection measures. Where real-world records are not available, controlled test records can be used to examine the expected system behavior.

3.2.2 Price-Prediction Dataset

The price-reference module uses a synthetically generated dataset containing 1,000 samples. The dataset includes features such as district, paddy type, season, and quantity, with a generated price value serving as the prediction target. Publicly available paddy-price information published by the Paddy Marketing Board (PMB) was considered as a real-world reference when designing the synthetic price data. Figure 3 presents an example of the PMB price information.


Ministry of Agriculture, Livestock, Lands and Irrigation

වර්ගය	නාඋ (සුදු)	නාඋ (රතු)	සම්බා	කිරි සම්බා
	සහතික පී (හොතමතය 14 %)	සහතික පී (හොතමතය 14%)	සහතික පී (හොතමතය 14%)	සහතික පී (හොතමතය 14 %)
සහතික පී මිලදෑ ගැනීම් 01ක අවම මිලදී ගැනීමේ මිල (රු.)	120	120	130	140

	කිරි සම්බා	සම්බා	නාඋ
2010/11 මහ	30.00	30.00	28.00
2011 යල	30.00	30.00	28.00
2011/12 මහ	30.00	30.00	28.00
2012 යල	30.00	30.00	28.00
2012/13 මහ	35.00	35.00	32.00
2013 යල	35.00	35.00	32.00
2013/14 මහ	35.00	35.00	32.00
2014 යල	35.00	35.00	32.00
2014/15 මහ	50.00	50.00	45.00
2015 යල	50.00	50.00	45.00
2015/16 මහ	50.00	41.00	38.00
2016 යල	50.00	41.00	38.00
2016/17 මහ	41.00	41.00	38.00
2017 යල	41.00	41.00	38.00
2017/18 මහ	41.00	41.00	38.00
2018 යල	41.00	41.00	38.00
2018/19 මහ	41.00	41.00	38.00
2019 යල	41.00	41.00	38.00
2019/20 මහ	50.00	50.00	50.00
2020 යල	50.00	50.00	50.00
2020/21 මහ	56.50	56.50	56.50
2022 යල	55.00	52.00	55.00
2021/22 මහ	95.00	92.00	90.00
2022 යල	130.00	125.00	120.00
2022/23 මහ	130.00	125.00	120.00
2023 යල	110.00	100.00	95.00
2023/24 මහ	130.00	120.00	105.00
2024 යල	130.00	115.00	105.00
2024/25 මහ	132.00	125.00	120.00
2025 yala	132.00	125.00	120.00
2025/26 maha	140.00	130.00	120.00

Figure 3 : Paddy Marketing Board price information

The model uses a linear-regression approach implemented through scikit-learn. A simplified federated-learning-style process simulates four district clients and

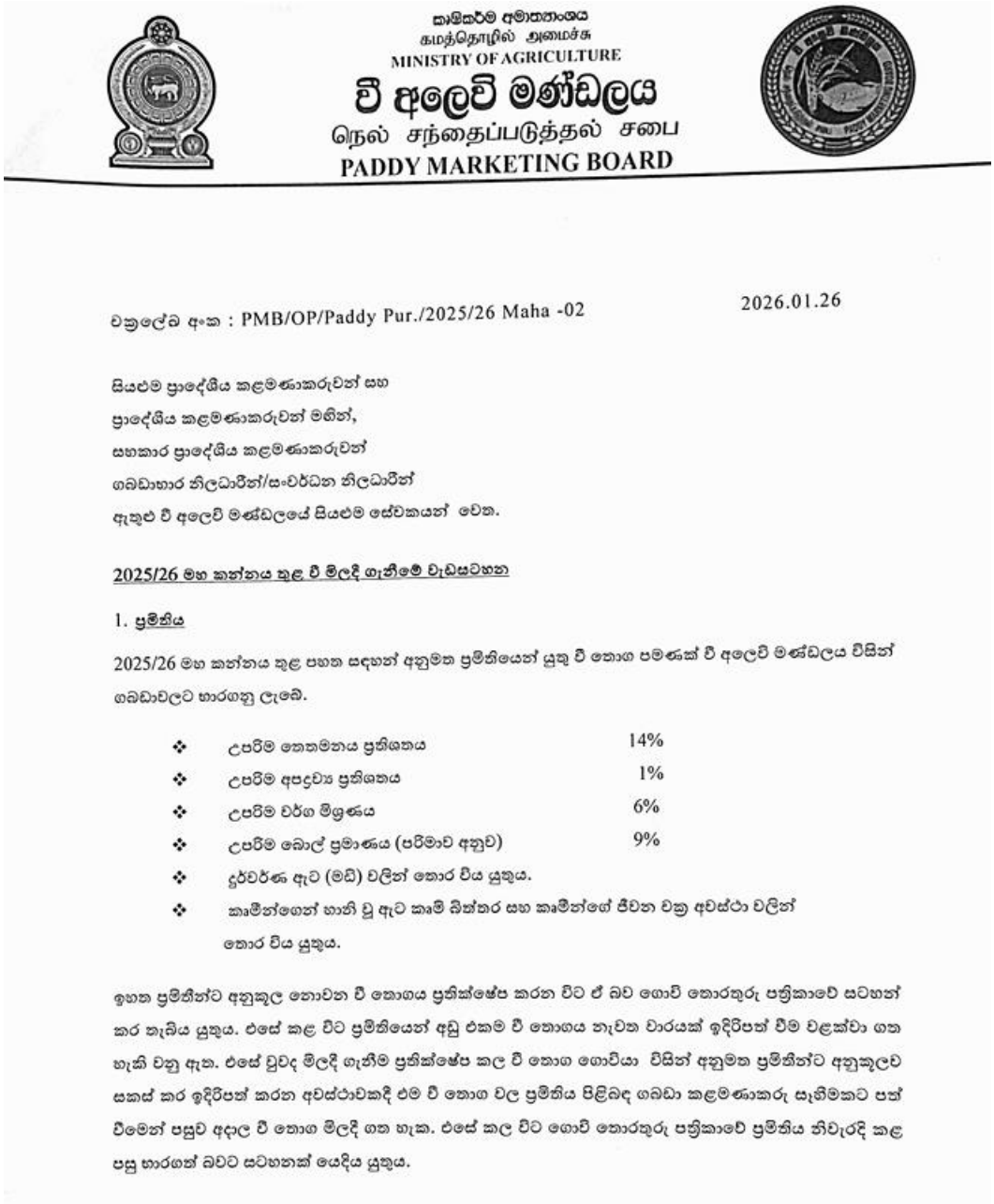
aggregates model parameters over multiple rounds. Synthetic data enables the training and prediction pipeline to be tested under controlled conditions. However, because the training samples are generated rather than collected from actual farmer–miller transactions, the results cannot establish the model's accuracy in predicting real-world paddy-market prices.

3.2.3 Knowledge Base for the RAG Assistant

The RAG assistant uses a curated collection of 32 knowledge records, comprising 15 PMB-related records and 17 marketplace-related records, according to the implementation description. The records provide contextual information for answering questions about supported marketplace functions and relevant agricultural market information.

The retrieval process uses multilingual text embeddings and a FAISS similarity index to identify potentially relevant records. Retrieved content is then supplied to the language model to generate a response.

The assistant's output depends on the coverage, correctness, and freshness of the supplied knowledge. Since the system does not continuously retrieve live PMB information, it cannot guarantee that every response reflects the latest official information.



කෘෂිකර්ම අමාත්‍යාංශය
கமத்தொழில் அமைச்சு
MINISTRY OF AGRICULTURE
වී අලෙවි මණ්ඩලය
நெல் சந்தைப்படுத்தல் சபை
PADDY MARKETING BOARD

චක්‍රලේඛ අංක : PMB/OP/Paddy Pur./2025/26 Maha -02 2026.01.26

සියළුම ප්‍රාදේශීය කළමණාකරුවන් සහ
ප්‍රාදේශීය කළමණාකරුවන් මඟින්,
සහකාර ප්‍රාදේශීය කළමණාකරුවන්
ගබඩාහාර නිලධාරීන්/සංවර්ධන නිලධාරීන්
ඇතුළු වී අලෙවි මණ්ඩලයේ සියළුම සේවකයන් වෙත.

2025/26 මහ කන්නය තුළ වී මිලදී ගැනීමේ වැඩසටහන

1. ප්‍රමිතිය

2025/26 මහ කන්නය තුළ පහත සඳහන් අනුමත ප්‍රමිතියෙන් යුතු වී තොග පමණක් වී අලෙවි මණ්ඩලය විසින් ගබඩාවලට භාරගනු ලැබේ.

- උපරිම තෙතමනය ප්‍රතිශතය 14%
- උපරිම අපද්‍රව්‍ය ප්‍රතිශතය 1%
- උපරිම වර්ග මිශ්‍රණය 6%
- උපරිම බොල් ප්‍රමාණය (පරිමාව අනුව) 9%
- දූර්වර්ණ ඇට (මඩි) වලින් තොර විය යුතුය.
- කෘමීන්ගෙන් හානි වූ ඇට කෘමි බිත්තර සහ කෘමීන්ගේ ජීවන චක්‍ර අවස්ථා වලින් තොර විය යුතුය.

ඉහත ප්‍රමිතීන්ට අනුකූල නොවන වී තොගය ප්‍රතික්ෂේප කරන විට ඒ බව ගොවි තොරතුරු පත්‍රිකාවේ සටහන් කර තැබිය යුතුය. එසේ කළ විට ප්‍රමිතියෙන් අඩු එකම වී තොගය නැවත වාරයක් ඉදිරිපත් වීම වළක්වා ගත හැකි වනු ඇත. එසේ වුවද මිලදී ගැනීම ප්‍රතික්ෂේප කල වී තොග ගොවියා විසින් අනුමත ප්‍රමිතීන්ට අනුකූලව සකස් කර ඉදිරිපත් කරන අවස්ථාවකදී එම වී තොග වල ප්‍රමිතිය පිළිබඳ ගබඩා කළමණාකරු සැහීමකට පත් වීමෙන් පසුව අදාල වී තොග මිලදී ගත හැක. එසේ කල විට ගොවි තොරතුරු පත්‍රිකාවේ ප්‍රමිතිය නිවැරදි කළ පසු භාරගත් බවට සටහනක් යෙදිය යුතුය.

Figure 4 : Main source PDF for RAG assistant

3.2.4 Testing Materials

Controlled test cases are used to examine the expected behavior of the system. These include valid and invalid listing inputs, compatible and incompatible harvest–demand pairs, queries with relevant knowledge records, questions with insufficient supporting information, and negotiation scenarios with different offers and constraints.

The study does not claim to have recruited human participants unless user testing or interviews were actually conducted and documented. If no such study was performed, the evaluation should be described as software-based testing using controlled data and scenarios.

3.3 Experimental or study procedure

The frontend is developed using React Native and Expo, with navigation and user interactions supporting farmer and miller workflows. The Node.js and Express backend handles API requests, application logic, authentication, and database interactions. MongoDB is used to store marketplace records.

The AI-related functions are implemented through separate services or modules. The RAG engine retrieves contextual knowledge and generates answers. The price-reference module produces estimated prices. The matching mechanism ranks candidate harvest–demand combinations. The negotiation service coordinates farmer and miller agents and validates their proposed decisions.

The frontend communicates with the backend API, while the backend interacts with the database and the relevant AI services. This modular structure allows individual functions to be tested separately before evaluating the complete marketplace workflow.

3.3.2 Functional Testing

Functional testing examines whether each implemented feature behaves according to its defined requirements. The tests cover user authentication, harvest and demand management, matching, notifications, and negotiation-related operations.

For example, listing tests examine whether valid records can be created and updated and whether invalid inputs are rejected appropriately. Matching tests examine whether the system identifies compatible records and applies the intended scoring rules. Negotiation tests examine whether generated offers comply with the configured constraints and whether the process terminates under the defined conditions.

Integration testing is used to check whether information flows correctly between the frontend, backend, database, and AI services. Error handling is also considered, including cases in which a service returns an error or the required information is unavailable.

Only tests that have actually been executed should be reported as completed. Test cases that have been designed but not run should be identified as planned or pending evaluation.

3.3.3 RAG Evaluation Procedure

The RAG assistant is evaluated using a collection of representative questions related to marketplace operations and the available PMB knowledge records. The test questions should cover direct factual questions, questions requiring the retrieval of relevant contextual information, multilingual or Singlish-style queries where supported, and questions for which the knowledge base does not contain a sufficient answer.

For each query, the retrieved records and generated response are examined. The evaluation considers whether the retrieved information is relevant, whether the response addresses the question, and whether factual statements are supported by the available context.

Questions with insufficient evidence are particularly important because they help establish whether the assistant appropriately communicates uncertainty rather than generating unsupported information. The results should be recorded using a consistent test set and clearly defined evaluation criteria.

3.3.4 Price-Reference Model Evaluation

The price-reference model is examined using the available synthetic dataset and a clearly documented evaluation procedure. The model's predicted values are compared with the corresponding reference targets using appropriate regression metrics.

Where training and evaluation subsets are used, their separation must be documented to reduce the risk of evaluating the model on information already used during training.

Any preprocessing, feature encoding, training configuration, and model-saving procedure should also be recorded.

The simplified federated-learning-style workflow is examined to confirm that client updates are generated and aggregated as intended. Because the data and clients are simulated, the evaluation focuses on the operation of the prototype and its measured performance on the available dataset. Claims about actual market accuracy or privacy protection require additional evidence.

3.3.5 Matching and Negotiation Evaluation

The matching mechanism is tested using controlled harvest and demand records with different combinations of district, paddy type, price, and quantity. The expected compatibility and score are calculated according to the implemented rules and compared with the system output.

The negotiation service is tested using scenarios that include offers within the permitted limits, offers that violate a constraint, counteroffers, potential agreement, and termination at the configured round limit. These scenarios are used to examine the correctness of the validator and the consistency of the negotiation workflow.

Finally, end-to-end scenarios examine how a user can move from listing management to matching and negotiation. These tests help identify problems that may not be visible when each module is evaluated independently.

To empirically evaluate the behavior and reliability of the negotiation module, a series of automated simulation runs were executed.

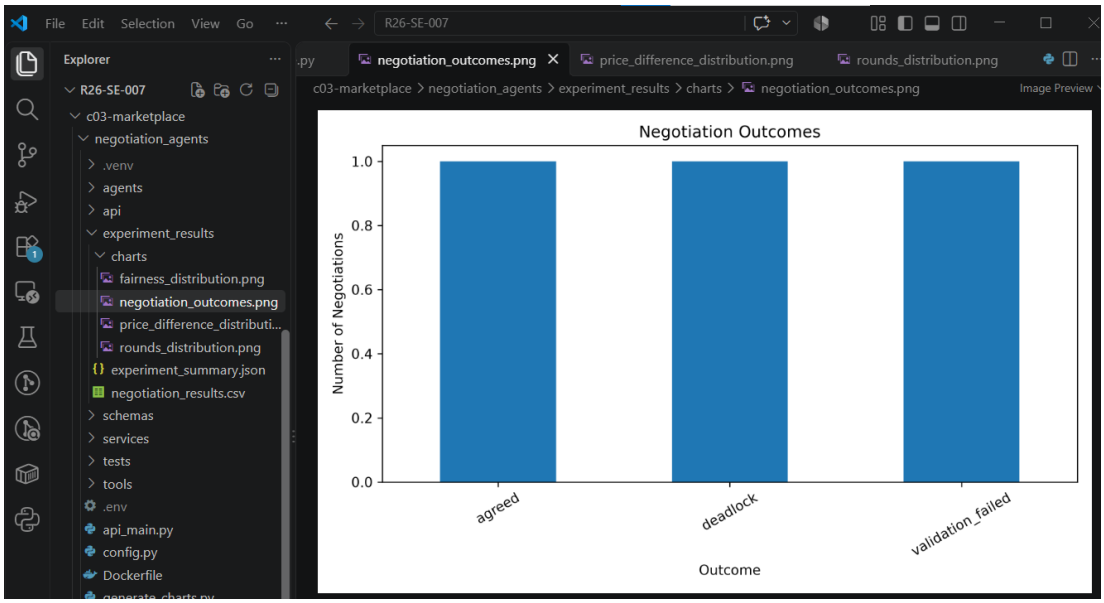


Figure 5 : Analysis of Negotiation Outcomes

The system's robust handling of distinct protocol states is illustrated in Figure 5. The even distribution among agreed, deadlock, and validation_failed outcomes confirms that the system successfully identifies valid concessions, terminates when agents reach an impasse without violating constraints, and correctly triggers the validator to reject illegal offers.

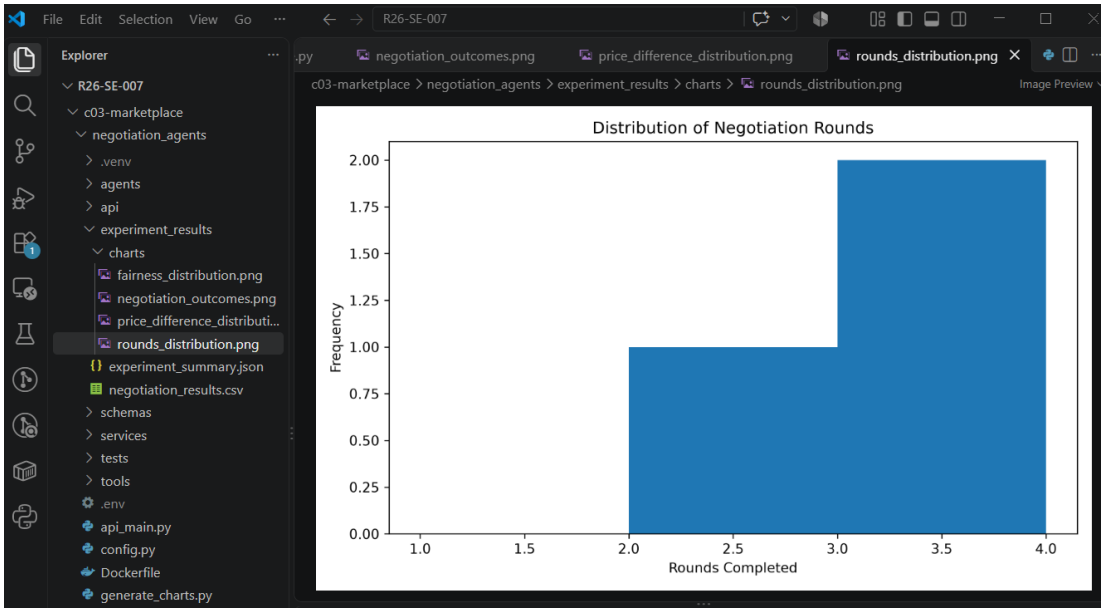


Figure 6 : Analysis of Conversation Depth

Figure 6 displays the number of rounds completed before a final state was reached. The distribution indicates that while some negotiations settled quickly (around 2 rounds), the majority utilized 3 to 4 rounds of counter-offering. This demonstrates the agents' capacity to sustain a multi-turn dialogue to resolve conflicting constraints before hit or miss termination occurs.

3.4 Measures and evaluation metrics

Different evaluation measures are used because the C03 modules perform different tasks. A single metric cannot adequately represent the performance of the entire marketplace.

Component	**Evaluation measures**	**Purpose**
Functional operations	Number of passed and failed test cases	Assess whether implemented features behave as expected
RAG assistant	Retrieval relevance, answer relevance, factual support, and appropriate handling of insufficient knowledge	Assess the quality and reliability of information assistance
Price-reference model	MAE, RMSE, R^2, and MAPE where mathematically appropriate	Measure prediction error and the relationship between predicted and reference values
Matching mechanism	Expected-score agreement, compatibility correctness, and ranking behavior	Assess whether the implemented rules are applied correctly
Negotiation service	Constraint compliance, agreement rate, rounds per negotiation, and correct termination	Assess negotiation workflow behavior

Integration	Successful end-to-end scenarios and recorded integration failures	Assess interactions between the frontend, backend, database, and AI modules

Figure 7 : Marketplace Evaluation measures

3.4.1 Regression Metrics

Mean Absolute Error (MAE) measures the average absolute difference between predicted and reference values. Root Mean Squared Error (RMSE) gives greater weight to larger prediction errors because the differences are squared before averaging. The coefficient of determination (R^2) indicates how well the model accounts for variation in the evaluation targets relative to a mean-based reference model.

Mean Absolute Percentage Error (MAPE) expresses prediction errors as percentages of the reference values. However, MAPE can become undefined or misleading when reference values are zero or close to zero. It should therefore be reported only when appropriate for the dataset.

These metrics describe different aspects of prediction performance. They should be interpreted together and alongside the characteristics of the evaluation data. Strong results on synthetic data do not establish equivalent performance on actual paddy-market prices.

3.4.2 RAG Evaluation Measures

Retrieval relevance examines whether the retrieved records contain information relevant to a query. Answer relevance examines whether the generated response addresses the question. Factual support examines whether the response is consistent with the retrieved material.

The evaluation can use a predefined answer key or a documented human-review rubric. If automated or model-assisted scoring is used, the method and its limitations should be stated. The number of test questions and the criteria for judging an answer as correct or incorrect should also be reported.

3.4.3 Matching and Negotiation Measures

For matching, expected-score agreement measures whether the system calculates scores according to the defined rules. Compatibility correctness examines whether the returned candidates satisfy the intended conditions. If a labelled evaluation dataset is available, ranking measures can also be considered.

For negotiation, constraint compliance measures whether accepted decisions respect the configured rules. Agreement rate is the proportion of completed test negotiations that reach an agreement under the test protocol. The number of rounds indicates how many interactions are required before termination. These measures describe the behavior of the implemented negotiation process; they do not independently demonstrate fairness or commercial success.

3.5 Data analysis

The collected evaluation results are analyzed using quantitative summaries and qualitative examination of system behavior.

Quantitative analysis is used for the numerical results produced by the price-reference model and the controlled software tests. Regression metrics are calculated from the available reference and predicted values. Functional tests are summarized by reporting the number of successful and unsuccessful cases, together with the observed failure types. Matching results are compared with the expected scores and compatibility rules. Negotiation results are summarized using the defined measures, such as constraint compliance and agreement rate.

Qualitative analysis is used to investigate errors that cannot be adequately explained by a single numerical value. For the RAG assistant, this includes reviewing queries for which relevant knowledge was not retrieved or the generated response was not sufficiently supported. For matching, it includes examining cases where the ranking does not reflect an expected compatibility condition. For negotiation, it includes reviewing scenarios in which an agent generates an invalid proposal or the workflow terminates unexpectedly.

The results are interpreted in relation to the purpose of each module. For example, a low prediction error on synthetic data indicates performance against the synthetic targets, not necessarily accuracy in a real market. A high matching score indicates compatibility under the implemented rules, not a guaranteed transaction. A successful negotiation test demonstrates correct behavior under the tested conditions, not that every real-world negotiation would reach a fair agreement.

Where comparisons between algorithms or configurations are performed, the same evaluation data and consistent procedures should be used. Any differences should be reported with the relevant dataset, configuration, and limitations. The analysis should avoid unsupported claims of improvement when no appropriate baseline or repeated evaluation has been conducted.

The findings from the separate modules are then considered together to assess the overall reliability of the marketplace workflow. This analysis helps identify which functions have demonstrated correct operation, which have measurable performance evidence, and which require further research or real-world validation.

3.6 Ethics privacy security and responsible AI

The C03 marketplace handles information associated with farmers, millers, listings, offers, and negotiation activities. Ethical and responsible development therefore requires attention to confidentiality, access control, data minimization, transparency, and the limitations of AI-generated outputs.

3.6.1 Data Privacy

The application should collect and process only the information needed to support its intended marketplace functions. Personal and commercial information should not be exposed to users who do not require access to it. Authentication and authorization mechanisms help restrict access to protected operations.

The RAG assistant should receive only the contextual information needed to answer a query. Private negotiation limits and other sensitive fields should not be unnecessarily included in prompts or returned in responses. The actual implementation must be

inspected to confirm what information is passed to AI services and what is stored in database records.

The use of synthetic data for the price-model prototype reduces the risk of exposing real farmer records during the described model evaluation. However, synthetic data does not eliminate privacy risks associated with user accounts or real marketplace operations.

3.6.2 Security

The backend uses authentication and access-control mechanisms to restrict protected operations. Password handling, token validation, input validation, and authorization checks are important safeguards for a marketplace that manages user records and trading information.

External service credentials, including API keys, should be stored in environment variables or an appropriate secret-management mechanism and must not be committed to a public repository. Sensitive information should be excluded from logs and error responses. Communication between deployed services should also be protected according to the deployment environment.

Security testing should examine unauthorized access, invalid input, inappropriate access to another user's records, and attempts to bypass role-specific restrictions. The presence of authentication alone does not establish that the entire application is secure; the controls must be tested.

3.6.3 Federated Learning and Privacy Claims

Federated learning is explored as a privacy-oriented training approach because it can reduce the need to transfer raw client data to a central training server. Nevertheless, model updates can still expose information under some conditions, and the degree of protection depends on the implementation.

The current marketplace price-reference prototype uses generated data and simulated clients. It therefore demonstrates a simplified training and aggregation process rather than validated privacy protection for real farmer data. The thesis does not claim that the current prototype provides differential privacy, secure aggregation, or a formal

privacy guarantee unless these mechanisms have actually been implemented and evaluated.

3.6.4 Responsible AI and Human Oversight

AI-generated answers and negotiation proposals can be incorrect, incomplete, or inconsistent with a user's circumstances. The RAG assistant should be treated as an information-support feature, and its answers should be understood in relation to the knowledge records available to it.

Similarly, predicted prices should be labelled as estimates rather than official prices or guaranteed selling prices. Matching scores should be explained as results of predefined rules. Negotiation agents should operate within explicit constraints, and users should retain control over real-world decisions and transaction confirmation.

The marketplace should also avoid presenting AI outputs as objective or universally fair without supporting evidence. Evaluation results should be reported transparently, including failures and limitations. This is especially important when users may rely on the system for decisions that affect their income or business activities.

3.7 Reproducibility and research artefacts

Reproducibility is important because it allows the implementation and evaluation process to be inspected, repeated, and extended by other researchers. The C03 project should maintain a clear record of its architecture, data sources, model configuration, test cases, and evaluation procedures.

The principal research artefacts include:

- **Source code:** the marketplace frontend, Express backend, RAG implementation, price-reference model, and negotiation service.
- **Configuration documentation:** service dependencies, runtime requirements, environment-variable names, API endpoints, and service startup instructions. Secret values must not be included.

- **Dataset documentation:** the origin, generation procedure, features, target variable, and limitations of the synthetic price dataset.
- **Knowledge-based documentation:** the structure and provenance of the curated RAG records, together with the embedding model and retrieval settings.
- **Model and training documentation:** the algorithm, simulated client arrangement, aggregation procedure, training configuration, and model-evaluation method.
- **Test artefacts:** functional test cases, RAG query sets, matching scenarios, negotiation scenarios, expected outcomes, and recorded results.
- **Evaluation outputs:** regression metrics, functional test summaries, matching results, negotiation measures, and documented errors.
- **Version-control history:** source-code changes and the versions used for the reported evaluation.

Where possible, the evaluation should record software versions, model settings, seeds for data generation, and the configuration used to obtain the reported results. The exact version of each dependency should be captured in the relevant dependency files. If external AI services are used, the model identifier and relevant parameters should also be recorded. Reproducibility may be limited by external API availability, model updates, hardware differences, and differences in runtime environments. These limitations should be documented rather than assuming that every generated response will be identical across executions.

Finally, the published thesis and any accompanying repository should exclude API keys, passwords, access tokens, and other confidential information. Data or records that contain personal or commercially sensitive information should only be shared when authorized and appropriately protected.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The Digital Goviya project was designed to address challenges associated with information sharing, decision-making, and coordination within Sri Lanka's paddy supply chain. The overall solution consists of four complementary components, AI-driven paddy price forecasting, digital farming, an AI-powered farmer-miller marketplace, and warehouse coordination with privacy-related mechanisms. Within this ecosystem, this component focuses on improving the interaction between paddy farmers and rice millers by providing a digital environment for listing management, partner discovery, price-reference support, and negotiation.

The requirements for marketplace were identified based on the intended functionality of the component, its role within the overall project, and the technical constraints of the implemented prototype.

4.1.1 Functional Requirements

The functional requirements describe the operations that the marketplace should support.

ID	Functional requirement	Description
FR01	User authentication	Allow farmers and millers to register and log in using role-based accounts.
FR02	Harvest management	Allow farmers to create, view, update, and manage paddy harvest listings.
FR03	Demand management	Allow millers to publish and manage paddy purchasing requirements.
FR04	Marketplace matching	Identify potentially compatible harvest and demand listings using predefined matching criteria.

FR05	Price-reference prediction	Provide an estimated paddy price to support marketplace decisions.
FR06	RAG-based assistance	Provide contextual responses using a curated knowledge base containing marketplace and public agricultural information.
FR07	Partner connections	Support the discovery of potential trading partners and management of partner connections.
FR08	Match selection	Allow users to select a compatible harvest–demand pair and submit it for consideration by the other party.
FR09	AI-assisted negotiation	Support farmer–miller negotiation through two role-specific AI agents operating within defined constraints.
FR10	Notifications	Inform users about relevant marketplace activities and status changes.
FR11	Contact coordination	Support contact requests after an agreement has been reached.
FR12	Transaction status management	Allow users to manually update listing status to reflect outcomes such as sold or fulfilled.
FR13	Multilingual interface	Support English and Sinhala to improve accessibility for users.
FR14	Dashboard and summaries	Display relevant listing, matching, and marketplace activity information.

Table 4 : Functional Requirement

4.1.2 Non-Functional Requirements

Non-functional requirements define the expected quality and operational characteristics of the system.

ID	Requirement	Design consideration
NFR01	Security	Use authenticated access, password hashing, and token-based authorization.
NFR02	Privacy	Avoid unnecessarily exposing private user information and negotiation limits in marketplace-facing responses.
NFR03	Usability	Provide role-oriented screens and understandable labels for farmers and millers.
NFR04	Maintainability	Separate frontend, backend, database, and AI-related services into manageable modules.
NFR05	Reliability	Validate requests, handle service errors, and apply listing and negotiation status checks.
NFR06	Interoperability	Allow the mobile client and backend services to exchange data through defined API interfaces.
NFR07	Explainability	Present matching scores as rule-based compatibility indicators and predicted prices as estimates rather than guaranteed outcomes.
NFR08	Extensibility	Allow the knowledge base, matching rules, and AI service implementations to be extended in future versions.

Table 5 : Non-Functional Requirements

These requirements guided the implementation of Marketplace component. However, the presence of a requirement does not by itself establish that the system achieves a particular level of quality. Security, usability, reliability, and performance claims

should therefore be supported by the relevant tests and evaluation results in the subsequent chapters.

4.2 Component architecture within the complete solution

Digital Goviya is organized as a multi-component research solution in which each component addresses a different aspect of the paddy supply chain. The components are intended to complement one another rather than duplicate the same responsibilities.

4.2.1 Overall Solution Architecture

The four components have the following responsibilities:

- **C01: AI-Driven Paddy Price Forecasting:**

  Focuses on estimating paddy prices using forecasting techniques.

- **C02: Digital Farming:**

  Focuses on farming-related digital monitoring and decision-support capabilities.

- **C03: AI-Powered Farmer-Miller Marketplace:**

  Provides listing management, matching, contextual assistance, price-reference support, and AI- negotiation.

- **C04: Warehouse Coordination and Privacy-Related Mechanisms:**

  Addresses warehouse-related coordination and security or privacy mechanisms within the wider solution.

C03 serves as the marketplace-facing component of the ecosystem. It concentrates on the interaction between farmers and millers and provides decision-support functionality for potential transactions.

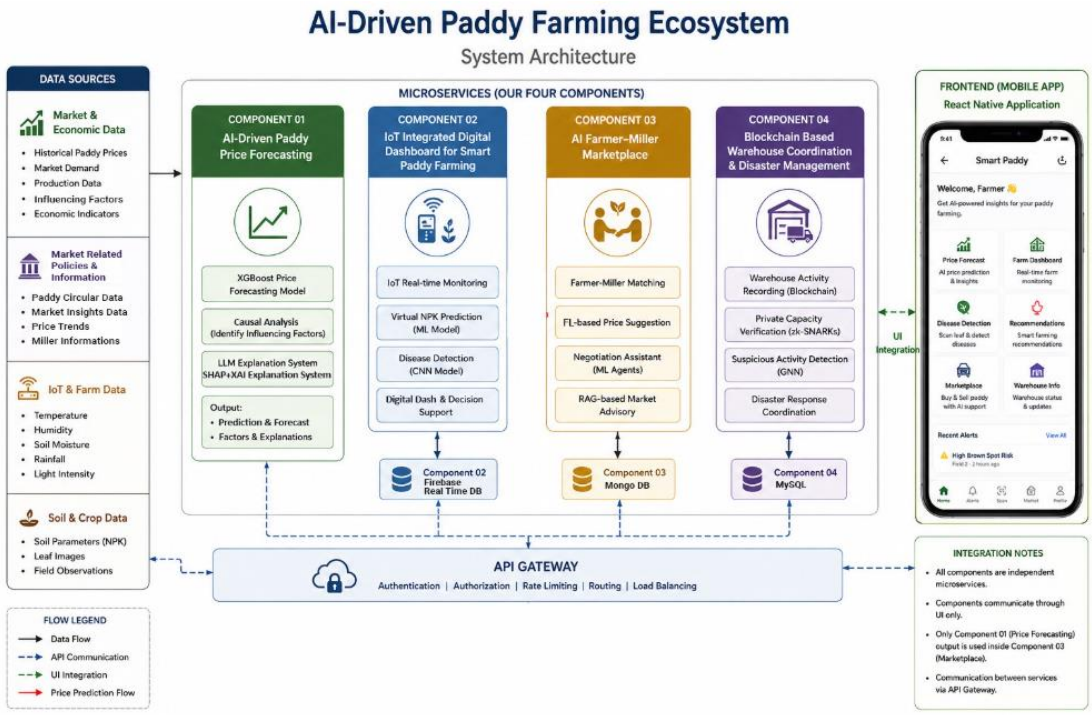


Figure 8 : Digital Goviya Overall System Architecture

4.2.2 Internal Architecture of AI-Powered Farmer-Miller Marketplace

The Farmer-Miller marketplace follows a layered architecture that separates user interaction, business operations, data persistence, and AI-related processing.

Presentation layer: The mobile application is developed using React Native and Expo, with Expo Router supporting screen navigation. It provides the user interface for authentication, harvest and demand listings, matching, connections, notifications, negotiation, dashboards, and language selection.

Backend application layer: A Node.js and Express 4 backend exposes HTTP API endpoints and coordinates application operations. It handles authentication, marketplace requests, listing management, match selection, and communication with supporting services. JWT-based authentication and bcrypt password hashing are used within the authentication implementation.

Data layer: MongoDB, accessed through Mongoose, stores application information such as user profiles, harvest listings, demand listings, partner relationships,

notifications, and negotiation-related records. The database supports the persistence of marketplace information between user sessions.

AI and decision-support layer: This layer contains three principal capabilities:

1. The price-reference module uses a simplified federated-learning-style process to generate estimated paddy prices.
2. The retrieval-augmented generation (RAG) assistant retrieves relevant information from a curated knowledge base and uses a language model to formulate contextual responses.
3. The negotiation service uses two role-specific language-model agents, with validation rules applied to their proposed actions.

The Python RAG and price-prediction modules support the Express backend. The negotiation functionality is provided through a separate Python FastAPI service, which the Express backend calls through HTTP requests.

This arrangement separates the marketplace's main application logic from AI-related processing. It also allows individual services to be modified or evaluated without requiring all responsibilities to be implemented within a single application module.

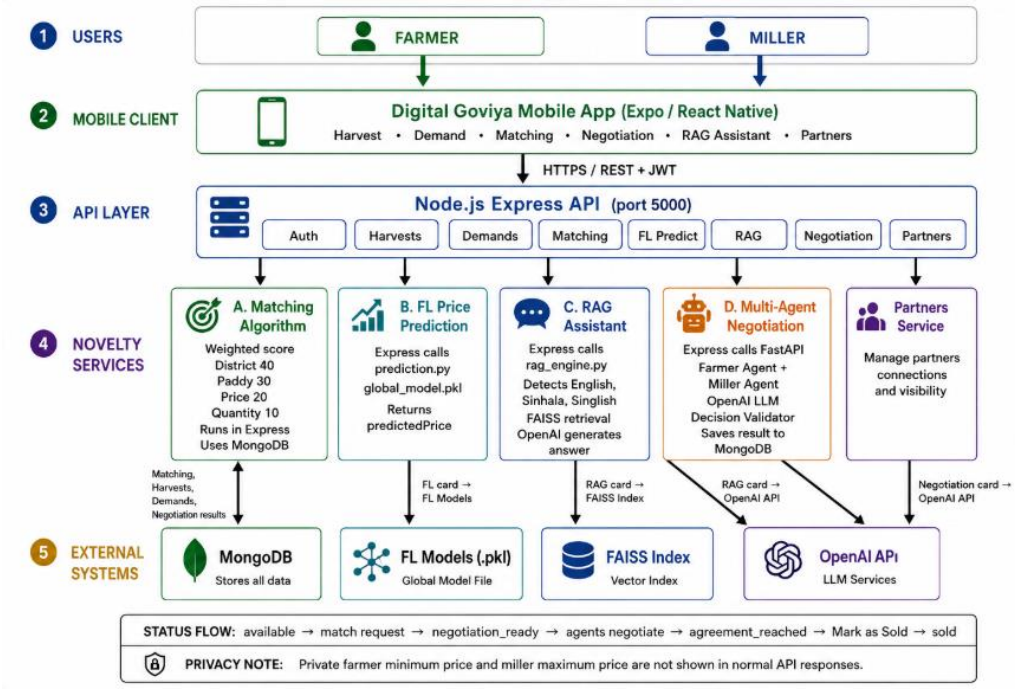


Figure 9 : AI-Powered Farmer-Miller Marketplace System Diagram

4.2.3 Interaction Between Architectural Components

When a user performs an operation, the mobile application sends a request to the Express backend. The backend validates the request, applies the relevant application rules, and reads or updates the required database records. For operations involving AI functionality, the backend invokes the appropriate supporting module or service.

For example, a user requesting marketplace matches triggers the matching logic, which compares eligible harvest and demand listings. A user requesting contextual guidance triggers the RAG workflow, while a negotiation request is passed to the separate negotiation service. Results are returned through the backend to the mobile application for display.

Although C03 is part of a four-component research project, its implemented internal services should not be confused with a fully integrated deployment of every group component. Any direct API connections or automated data exchange between C03 and C01, C02, or C04 should be described only where those integrations have actually been implemented.

4.3 Design alternatives and decisions

Several design choices were made to balance the research objectives, implementation complexity, available resources, and the need to demonstrate multiple decision-support approaches within one marketplace prototype.

4.3.1 Mobile Application Technology

A mobile application was selected to provide access to marketplace functions through a single interface. React Native and Expo were used to support cross-platform application development and simplify development and testing. This approach was preferred over developing separate native applications for Android and iOS, which would require maintaining distinct platform-specific implementations.

4.3.2 Backend Architecture

Node.js and Express were selected for the primary application backend because they support HTTP API development and communication with the mobile application and supporting services. Python was retained for the AI-related modules because the

selected machine-learning, retrieval, and model-serving libraries are available within the Python ecosystem.

Rather than placing every function within a single backend process, the negotiation functionality was implemented as a separate FastAPI service. This separation provides a clearer boundary around negotiation logic and allows the service to be tested independently. The trade-off is that communication between services introduces additional configuration, network dependencies, and error-handling requirements.

4.3.3 Database Selection

MongoDB was selected as the primary data store. Its document-oriented structure supports records such as user profiles, harvest listings, demand listings, and negotiation histories, which may contain different fields depending on the record type and lifecycle stage.

Mongoose provides schemas and data-access operations for the Node.js backend. This design supports application-level validation and modular data access. Nevertheless, database constraints and backend validation must still be applied carefully to prevent inconsistent listing and match states.

4.3.4 RAG-Based Knowledge Assistance

A conventional question-answering system based only on a language model could produce responses without explicitly retrieving information from the marketplace knowledge base. A retrieval-augmented generation approach was selected to provide contextual information from curated records.

The implemented RAG pipeline uses multilingual sentence embeddings, a FAISS vector index, and OpenAI's gpt-4.1-mini model. Retrieved records are provided as context to help generate relevant responses. The knowledge base contains 32 curated records, 15 public agricultural or Paddy Marketing Board-related records and 17 marketplace records.

This approach was selected because it combines semantic retrieval with language generation. Its limitation is that the quality of the response depends on the coverage and accuracy of the indexed records and the relevance of retrieval. The current

implementation does not continuously crawl live public sources, so it should not be presented as a real-time agricultural information service.

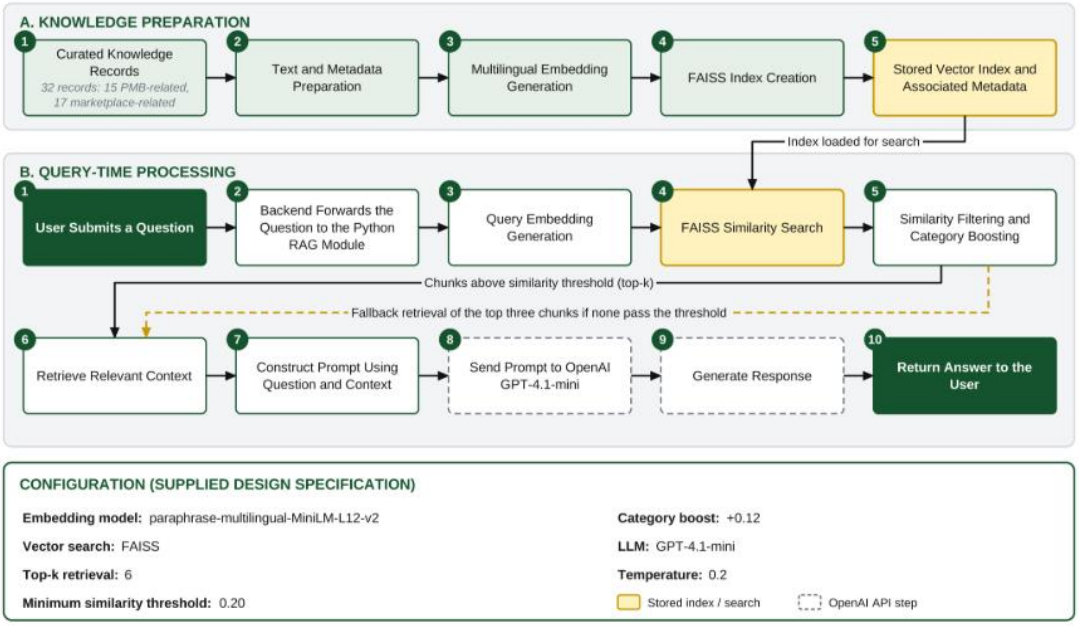


Figure 10 : RAG Architecture

4.3.5 Price-Reference Approach

A federated-learning-style approach was included to explore how model parameters can be combined across simulated district clients without implementing a conventional centralized training workflow for the prototype.

The current implementation uses an SGDRegressor model, synthetic training data, four simulated district clients, and five rounds of coefficient and intercept averaging. This was selected as a manageable demonstration of federated parameter aggregation within the project timeframe.

The implementation is a simplified in-process simulation. It does not represent a fully distributed federated-learning deployment, and its synthetic training data do not establish real-world forecasting accuracy. Consequently, the predicted price is presented as a decision-support reference rather than an official or guaranteed market price.

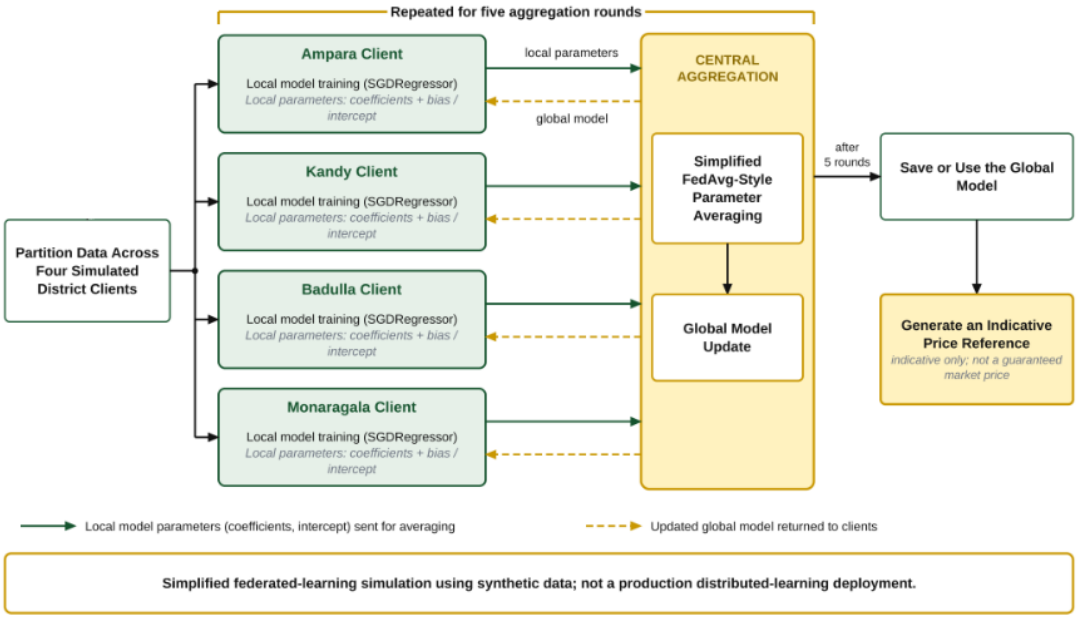


Figure 11 : Federated Learning Simulation workflow

4.3.6 Rule-Based Marketplace Matching

A trained matching model could require a representative dataset of completed transactions and reliable labels indicating successful matches. Since such a dataset was not established for the current implementation, a transparent weighted rule-based approach was selected.

The matching score combines district compatibility, paddy-type compatibility, price closeness, and quantity compatibility. The score is calculated out of 100 and used to categorize potential matches. This makes the decision process straightforward to inspect and modify.

The trade-off is that the system cannot automatically learn complex compatibility patterns from historical transactions. It also does not currently account for factors such as GPS distance, moisture content, grain quality, harvest season in the scoring formula, or mill capacity. The score should therefore be interpreted as a preliminary compatibility indicator rather than a probability that a transaction will succeed.

4.3.7 Multi-Agent Negotiation

A single language-model interaction could be used to suggest a price, but it would not explicitly represent the distinct negotiating positions of the farmer and miller. The implemented design therefore uses two role-specific agents with separate prompts and private minimum or maximum limits.

The negotiation service coordinates their sequential decisions and applies validation rules to the proposed actions. The process is bounded by a maximum of six rounds and includes checks for invalid offers and negotiation termination conditions.

This design provides a structured demonstration of bilateral negotiation. However, the agents are not independent human negotiators, and their decisions should not be treated as proof of economically optimal or fair outcomes. The implementation also does not provide cryptographic privacy guarantees simply because the agents have separate roles. The handling and storage of both parties' limits must be considered when describing the privacy properties of the system.

4.4 Detailed design

This section describes the principal internal workflows and decision rules used by the C03 marketplace.

4.4.1 User and Listing Management

The system distinguishes between farmer and miller roles. Following authentication, users can access the marketplace functions associated with their accounts. Farmers can create harvest listings describing available paddy, while millers can create demand listings describing their purchasing requirements.

Listing records are persisted in MongoDB. Their status is used to determine whether they remain eligible for marketplace matching. Listing status management is important because a listing that is already sold, fulfilled, or otherwise unavailable should not continue to be treated as an active opportunity.

4.4.2 Matching Algorithm Design

The matching module compares eligible harvest and demand listings in both directions: harvest listings can be compared against miller demands, and demand listings can be compared against available harvests.

The resulting score is used to classify a match as **Highly Recommended** for scores of 85 or above, **Recommended** for scores of 65 or above but below 85, and **Moderate** for scores below 65.

The system returns a limited list of high-ranking matches, with five as the default number. Where applicable, a connected trading partner can be brought into focus in the results. Partner connections affect visibility or prioritization; they do not independently increase the weighted compatibility score.

This algorithm is intentionally interpretable. However, the exact-match criteria may exclude potentially viable matches where district or paddy-type values differ slightly in representation. Similarly, the fixed score thresholds are design choices and require empirical evaluation before being treated as validated measures of trading suitability.

4.4.3 Price-Prediction Workflow

The price-reference module follows a simplified federated-learning-style process:

1. A generated dataset containing 1,000 generated records is prepared with features such as district, paddy type, season, and quantity.
2. The data are divided among four simulated district clients Ampara, Kandy, Badulla, and Monaragala.
3. Each client trains an SGDRegressor model locally within the simulation.
4. The model coefficients and intercepts are combined using an averaging procedure.
5. The aggregation process is repeated for five rounds.
6. The resulting model is used to produce an estimated paddy price for the marketplace workflow.

The workflow demonstrates parameter aggregation across simulated clients, but it does not establish that data remain physically distributed across separate devices or institutions. The generated dataset is synthetic, so model evaluation and claims about practical accuracy must reflect that limitation.

4.4.4 RAG Assistant Workflow

The RAG assistant is designed to answer questions using relevant information from the indexed knowledge base.

The workflow begins when the user submits a question through the application. The request is passed to the backend, which invokes the Python RAG module. The question is embedded using the multilingual sentence-transformer model and compared with vectors in the FAISS index.

The retrieval process selects up to six relevant records, applies the implemented similarity threshold and category adjustment, and uses a fallback retrieval procedure when necessary. The retrieved context is then provided to gpt-4.1-mini to generate a response.

The pipeline includes normalization and keyword handling intended to support Sinhala, Singlish, and English queries. However, multilingual input handling should not be interpreted as a guarantee that every answer will be accurate or naturally phrased in every language. The quality of the generated response depends on the retrieved evidence, the question, and the model's output.

4.4.5 Negotiation Workflow

The negotiation workflow begins after a harvest-demand match has been selected and the required approval or readiness conditions have been satisfied.

The FastAPI negotiation service coordinates two agents:

- **Farmer Agent:** Represents the farmer's negotiating position and evaluates proposed offers against the farmer-side constraints.
- **Miller Agent:** Represents the miller's negotiating position and evaluates proposed offers against the miller-side constraints.

The agents make sequential decisions through separate language-model calls. Their proposed actions are checked by the validation logic, which applies the configured minimum and maximum limits and determines whether the offer should be accepted, revised, or rejected. The process is limited to six rounds and includes termination checks to avoid indefinite negotiation.

A successful negotiation produces an agreement outcome within the system. However, **an agreement is not the same as a completed transaction**. Listing status is updated separately through the manual sold or fulfilled process, preserving the distinction between reaching an agreement and confirming that the trade has been completed.

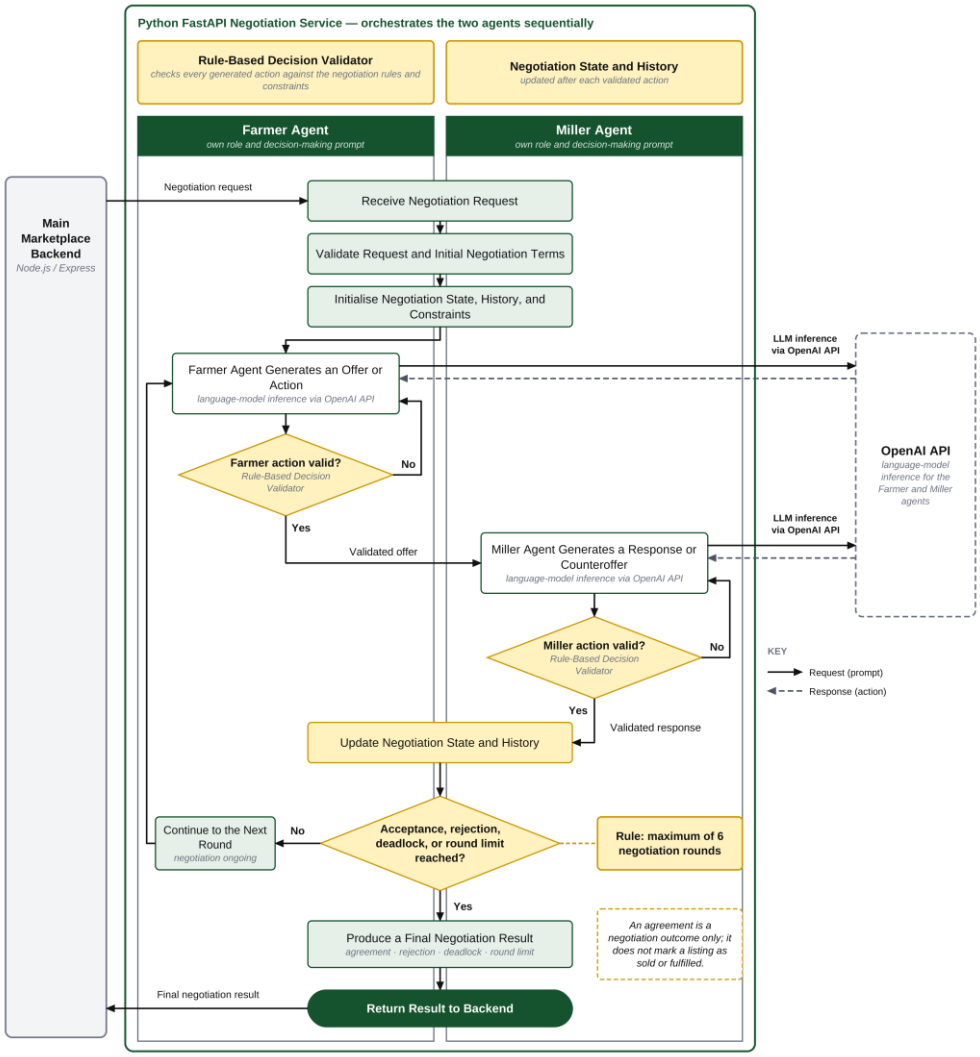


Figure 12 : Negotiation Agents Orchestration

4.4.6 Match Selection and Transaction Lifecycle

The match-selection workflow links a specific harvest listing with a specific demand listing. The backend creates a match-selection record and checks for an existing record with the same harvest-demand pair. A unique database constraint is used to help prevent duplicate selections.

The other party can respond to the selection. Once the required conditions are satisfied, the match can move to a state in which negotiation is permitted. After negotiation, the resulting agreement is recorded separately from the eventual transaction outcome.

The intended lifecycle can be represented as:

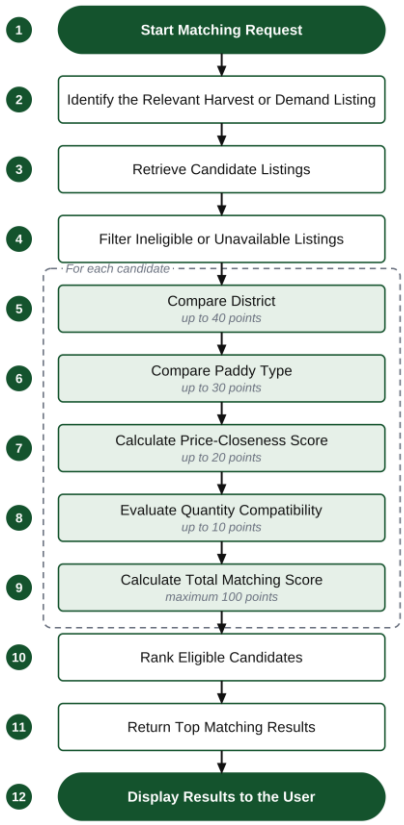


Figure 13 : Matching Life cycle

This lifecycle separates the discovery of a possible trading partner from the negotiation process and the final transaction status. It also reduces the risk of treating an AI-generated agreement as automatic evidence that a physical trade has taken place.

4.5 Standards and professional practice

The design and implementation of marketplace were guided by principles of modular software engineering, secure application development, responsible AI use, and transparent research reporting.

4.5.1 Software Engineering Practices

The application separates frontend presentation, backend business logic, database access, and AI-related services. This modular organization supports maintainability and makes it easier to test individual workflows. Defined HTTP interfaces are used for communication between the mobile application and backend, as well as between the Express backend and the separate negotiation service.

Request validation, status checks, and duplicate-match prevention are included in relevant workflows to reduce inconsistent application states. Dependency management and configuration separation should be maintained so that development and deployment environments can be reproduced.

4.5.2 Security and Privacy Practices

Authentication uses password hashing and JWT-based mechanisms. Protected operations should validate the user's identity and role before allowing access to account-specific or marketplace-specific data. Secrets, including API keys and database credentials, should be supplied through environment configuration rather than committed to source control.

Privacy-related design considerations include limiting the exposure of user profile details and private negotiating limits in user-facing responses. Nevertheless, these application-level measures should not be described as end-to-end encryption, differential privacy, or cryptographic protection unless the corresponding mechanisms have actually been implemented and evaluated.

4.5.3 Responsible AI and Decision Support

The Component AI functions have different roles and limitations. The RAG assistant generates responses based on retrieved knowledge-base records; the price module

provides an estimate, the matching module applies fixed compatibility rules; and the negotiation agents generate proposals subject to validation.

The interface and accompanying documentation should communicate these distinctions. A matching category does not guarantee that a transaction is suitable, a predicted price does not replace official price information, and an agent-generated agreement does not confirm that a transaction has been completed. Human users retain responsibility for their decisions and for manually recording the final transaction status.

4.5.4 Research Integrity and Reproducibility

The research report should accurately distinguish implemented functionality from planned work and should document the synthetic nature of the price-training data and simulated district clients. Evaluation results should be reported only when supported by executed tests or recorded experiments. The same principle applies to claims about model accuracy, user satisfaction, security, and performance.

Relevant research artefacts include source code, configuration templates without secrets, dependency information, the curated RAG knowledge base, data generation logic, model-training scripts, test cases, and evaluation outputs. Where possible, version information and execution instructions should be recorded to support repeatability.

The system design therefore emphasizes modularity, traceability, and responsible reporting while acknowledging that the current prototype remains a research implementation rather than a fully validated commercial marketplace.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The AI-powered Farmer-Miller Marketplace was implemented using a combination of mobile application technologies, backend frameworks, database services, and AI-related libraries. The technologies were selected to support the development of a modular application in which marketplace operations and AI-assisted functions could be implemented and maintained separately.

5.1.1 Development Environment

The application was developed in a Windows-based environment. The development setup included Node.js for the main backend, Python for machine-learning and AI-related modules, and a database service for storing marketplace information. The mobile interface was developed using React Native and Expo, allowing the application to be tested during development through supported mobile and web environments.

The project was organized into separate modules for the marketplace backend, price forecasting, negotiation, and mobile client. This organization helped separate the responsibilities of each module and provided a basis for independent development and testing.

5.1.2 Technology Stack

Technology	**Purpose within C03**
React Native	Development of the mobile application interface.
Expo and Expo Router	Mobile application development and screen navigation.
Node.js	Runtime environment for the primary backend.
Express 4	HTTP API endpoints and marketplace business logic.
MongoDB	Persistence of user, listing, matching, and related application records.

Mongoose	Database schemas and data-access operations.
JWT	Token-based authentication and authorization support.
bcrypt	Password hashing.
Python	Implementation of price prediction, RAG processing, and negotiation logic.
scikit-learn	Implementation of the SGDRegressor price-prediction model.
NumPy and pandas	Numerical processing and dataset preparation.
FAISS	Similarity search over indexed knowledge-base embeddings.
Sentence Transformers	Multilingual text embedding generation.
OpenAI gpt-4.1-mini	Language generation in the RAG assistant.
FastAPI	HTTP service interface for the negotiation module.

Table 6 : Technology Stack

The technology stack separates general marketplace operations from AI-related computation. The Express backend coordinates application requests, while Python modules perform specialized processing. The negotiation service runs separately through FastAPI, whereas the RAG and price-prediction modules are invoked through the backends' configured integration mechanisms.

The table represents the implementation described for the current research prototype. Exact library versions, operating-system details, and deployment configurations should be recorded from the actual development environment to support reproducibility.

5.2 Component implementation

The implementation of Marketplace component was organized around the principal marketplace functions: user management, listing management, matching, price-reference prediction, knowledge assistance, negotiation, and transaction coordination.

5.2.1 User Authentication and Marketplace Management

The authentication module supports access through farmer and miller accounts. Passwords are hashed using bcrypt, and JWTs are used to support authenticated API requests. The backend applies role and access checks to operations that require an authenticated user.

After authentication, farmers can manage harvest listings, while millers can manage demand listings. Listing records are stored in MongoDB through Mongoose-based data-access operations. These records contain the information required for marketplace discovery and matching.

The implementation also supports partner connections, notifications, dashboards, and related marketplace interactions. The listing lifecycle distinguishes between available listings and those that are no longer available for matching.

5.2.2 Price-Reference Prediction

The price-prediction module was implemented in Python using scikit-learn's SGDRegressor. Supporting libraries include NumPy and pandas for data preparation and numerical operations.

The current prototype uses a synthetic dataset of 1,000 records generated using a predefined formula and added noise. Four simulated district clients-Ampara, Kandy, Badulla, and Monaragala-participate in five rounds of local model training and parameter averaging.

After aggregation, the resulting model can generate a paddy price estimate for use in marketplace decision support. The estimate provides an additional reference when comparing potential offers; it does not represent an official guaranteed price.

Because the training dataset is synthetic, the implementation demonstrates the model workflow rather than establishing forecasting performance under real-world market conditions. This limitation is important when interpreting the evaluation results.

5.2.3 Retrieval-Augmented Generation Assistant

The RAG assistant was implemented using a Python retrieval pipeline. The knowledge base contains 32 curated records, comprising 15 public agricultural or Paddy Marketing Board-related records and 17 marketplace records.

The text is converted into multilingual embeddings using sentence-transformers/ paraphrase-multilingual-MiniLM-L12-v2. The embeddings are stored in a FAISS index, which supports similarity-based retrieval. When a user submits a question, the retrieval process identifies relevant records and supplies them as context to OpenAI's gpt-4.1-mini model.

The implementation retrieves up to six candidate records and applies the configured similarity threshold and category adjustment. A fallback retrieval procedure is used when the initial selection does not contain sufficient records. The response is then returned through the application's backend integration.

Sinhala, Singlish, and English input handling is supported through normalization and keyword-based processing. However, response quality depends on the knowledge base, retrieval relevance, and generated output. The current implementation does not continuously retrieve live information from external public websites.

5.2.4 Rule-Based Marketplace Matching

The matching module was implemented using a weighted scoring algorithm rather than a trained machine-learning classifier. It compares available farmer harvest listings with open miller demand listings and can perform the comparison in both directions.

The algorithm evaluates district compatibility, paddy-type compatibility, price closeness, and quantity compatibility. These criteria contribute a maximum of 40, 30, 20, and 10 points respectively, producing a score out of 100.

The score is used to classify results into Highly Recommended, Recommended, or Moderate categories. The module returns a limited number of matches, with five as

the default. Where applicable, a connected trading partner can be brought into focus without automatically increasing the compatibility score.

This implementation provides an interpretable basis for ranking listings. Its effectiveness depends on the selected criteria and thresholds, and the score should not be interpreted as a probability of successful trading.

5.2.5 Multi-Agent Negotiation

The negotiation functionality was implemented as a separate Python service using FastAPI. It contains two role-specific agents, a Farmer Agent and a Miller Agent. Both use the OpenAI's gpt-4.1-mini model, with separate prompts representing their respective negotiating positions.

The service coordinates sequential model calls and validates the proposed actions against configured negotiation constraints. The validation logic checks the applicable limits and determines whether an offer can be accepted, revised, or rejected. The negotiation process is bounded by a maximum of six rounds and includes termination checks.

The Express backend communicates with the negotiation service through HTTP requests. Separating this functionality from the main backend provides a defined boundary for negotiation processing and testing.

An agreement reached by the agents is recorded separately from transaction completion. The application allows the relevant user to manually update a listing as sold or fulfilled, rather than treating an AI-generated agreement as proof that the physical transaction has occurred.

5.3 Integration with the other components

Integration within Digital Goviya involves both communication among the internal modules of C03 and coordination with the project's other research components. These two forms of integration should be distinguished when describing the implemented system.

5.3.1 Integration Within Marketplace Component

The mobile application communicates with the Express backend through HTTP API requests. The backend handles the relevant business logic and interacts with MongoDB when application data must be retrieved or updated.

For AI-assisted functions, the backend invokes the relevant processing module. The RAG and price-prediction functionality is implemented in Python, while negotiation requests are sent to the separate FastAPI service. The returned outputs are processed by the backend and made available to the mobile application.

The principal internal communication paths are summarized below.

Source	**Destination**	**Purpose**
Expo mobile application	Express backend	Authentication and marketplace API requests.
Express backend	MongoDB	Reading and updating application records.
Express backend	Python RAG module	Retrieving knowledge-base context and generating answers.
Express backend	Price-prediction module	Obtaining estimated paddy prices.
Express backend	FastAPI negotiation service	Initiating and managing negotiation operations.
Negotiation service	Multi agent Orchestration and OpenAI API	Generating role-specific negotiating actions.
RAG module	FAISS index and embedding model	Retrieving relevant knowledge-base records.
RAG module	OpenAI API	Generating contextual answers from retrieved information.

Table 7 : Internal Communication Paths within component

These interfaces allow the modules to work together as part of the marketplace workflow. Their operational reliability depends on correct configuration, network accessibility, valid request and response formats, and suitable error handling.

5.3.2 Integration Within Other Components

The wider Digital Goviya project contains three additional components with different responsibilities, price forecasting in Component 01, digital farming in Component 02, and warehouse coordination and privacy-related mechanisms in Component 04.

Any direct integration with C01, C02, or C04 must be documented according to the actual interfaces, data formats, and workflows implemented. If a component is not directly connected to C03, it should be described as a related part of the overall solution rather than as an integrated service.

For the current report, the demonstrated internal C03 service interfaces should be documented separately from any group-level integration that remains planned or incomplete. This distinction helps maintain an accurate record of the individual component's contribution.

5.3.3 Integration Challenges

The architecture introduces several practical integration considerations. First, the mobile application must use a backend URL that is reachable from the device or environment in which it is running. A local address that works on the development computer may not be accessible from a physical phone or a remotely deployed application.

Second, the Express backend must be able to reach the FastAPI negotiation service and invoke the Python modules with the correct configuration. Service failures, invalid responses, and timeouts should be handled without causing the entire application to fail unexpectedly.

Finally, the use of external language-model services and local model inference introduces different configuration and operational requirements. API credentials must be protected.

5.4 Verification and software testing

Testing was considered at several levels to examine whether the implemented modules perform their intended functions and communicate correctly. The final report should distinguish between test cases that were designed, tests that were executed, and tests that passed. Only executed tests with recorded results should be presented as completed verification evidence.

5.4.1 Testing Approach

The following testing categories are relevant to Marketplace component:

Functional testing: Verifies whether authentication, listing management, matching, partner connections, notifications, and manual listing-status updates behave according to their requirements.

Integration testing: Examines communication between the mobile application, Express backend, database, RAG module, price-prediction module, and negotiation service.

AI-module testing: Evaluates retrieval relevance and response support for the RAG assistant, prediction errors for the price-reference model, matching-score correctness, and negotiation constraint compliance.

Error-handling testing: Checks invalid requests, unavailable services, missing or incorrect fields, unauthorized operations, duplicate match selections, and unsuccessful negotiation outcomes.

Usability and interface testing: Examines navigation, role-specific screens, language selection, and the clarity of information displayed to farmers and millers. Formal usability conclusions require an appropriate evaluation procedure and supporting evidence.

5.4.2 Functional and Integration Test Cases

The following table provides a test plan for documenting the verification process. The expected outcomes are design expectations.

Test ID	Test scenario	Expected result
TC01	Submit valid farmer or miller login credentials.	The user is authenticated and can access the appropriate interface.
TC02	Submit invalid login credentials.	Authentication is rejected and an appropriate error is returned.
TC03	Create a harvest or demand listing with valid data.	The listing is stored and displayed to the relevant user.
TC04	Submit incomplete or invalid listing data.	Validation prevents invalid data from being accepted.
TC05	Request matches for compatible active listings.	Eligible listings receive scores based on the configured matching rules.
TC06	Attempt to select the same harvest–demand pair more than once.	Duplicate selection is prevented by the implemented checks and database constraint.
TC07	Submit a representative question to the RAG assistant.	The module retrieves relevant knowledge-base records and returns a contextual response.
TC08	Request an estimated paddy price.	The price module returns a numerical estimate for the supplied input.
TC09	Start a negotiation between the two agents.	The service processes the negotiation within its configured round limit and constraints.
TC10	Attempt an operation without the required authorization.	The protected operation is rejected.

TC11	Mark a listing as sold or fulfilled manually.	The listing status is updated and the listing is excluded from subsequent matching where configured.
TC12	Test the mobile application when a supporting service is unavailable.	The application handles the failure without presenting an invalid successful result.

Table 8 : Functional Test Cases

5.5 Deployment and operation

The C03 deployment design separates the mobile client, the main backend, the database, and the supporting AI services. This allows each part to be configured according to its runtime requirements.

5.5.1 Service Configuration

During development, the Express backend runs on port 5000, while the FastAPI negotiation service runs on port 8000. The mobile application uses the configured API URL, supplied through the EXPO_PUBLIC_MARKETPLACE_API_URL environment variable.

The backend also requires database connection configuration and the relevant AI-service settings. The RAG and Multi-agent module uses an OpenAI API credential in the current implementation.

These settings must be provided through environment-specific configuration. Credentials and other secrets should not be embedded in source code or included in screenshots, public repositories, or the final report.

5.5.2 Development and Mobile Access

During development, the mobile application and backend may run on the same local network. However, the backend URL must be reachable from the device running the application. A localhost address on a physical phone refers to the phone itself, not the

development computer. Therefore, local device testing requires a suitable network-accessible address and compatible firewall and server configuration.

The mobile application's configuration must also be updated appropriately when moving from local development to a hosted environment. Successful access from a desktop browser alone does not establish that the backend is accessible from a physical mobile device.

5.5.3 Deployment Considerations

A production-style deployment requires the backend and supporting services to be hosted in an environment where they can communicate reliably. The database must be accessible to authorized services, API endpoints should use HTTPS, and credentials must be stored securely. CORS, network access, authentication, request validation, and error handling should be configured according to the deployment environment.

The RAG assistant requires its knowledge-base files and FAISS index, as well as access to the OpenAI API. These dependencies must be included in the deployment plan or replaced with equivalent hosted services where appropriate.

Docker can be used to package compatible services and standardize their runtime configuration. However, containerization should be reported as an implemented deployment feature only if the relevant container configurations have been created and tested. Similarly, a local development setup should not be described as a publicly deployed production system unless public hosting and operational verification have been completed.

5.5.4 Operational Monitoring and Maintenance

The marketplace requires ongoing attention to service availability, configuration correctness, database integrity, and the quality of AI outputs. Operational logs can help identify failed requests, inaccessible services, and unexpected workflow states. Logs should avoid unnecessarily recording passwords, API keys, private negotiation limits, or other sensitive information.

The knowledge base and FAISS index should be updated when curated information changes. Changes to the matching rules, synthetic-data generation process, model

configuration, or negotiation constraints should be version-controlled and tested before being incorporated into the active application.

The C03 prototype provides a modular foundation for demonstrating marketplace coordination and AI-assisted decision support. Its readiness for wider deployment depends on completing and verifying the necessary integration, security, reliability, and real-world evaluation activities.

6 RESULTS AND VALIDATION

6.1 Research results

The research focused on developing and evaluating an AI-powered marketplace to support coordination between paddy farmers and rice millers in Sri Lanka. The C03 component combines rule-based marketplace matching, price-reference prediction, retrieval-augmented generation (RAG), and multi-agent negotiation within a single application.

The research results should be interpreted according to the evidence available for each function. The matching algorithm and negotiation workflow can be evaluated through controlled scenarios, while the price-prediction module requires numerical evaluation and the RAG assistant requires assessment of retrieval relevance and answer quality.

6.1.1 Price-Prediction Results

The price-reference module was evaluated using a dataset of 1,000 samples and a saved global model. The recorded evaluation results are summarized in Table 6.1.

Table 6.1: Price-prediction evaluation results

Evaluation measure	Recorded result
Number of evaluation samples	1,000
Mean actual price	Rs. 124.81/kg
Mean predicted price	Rs. 123.86/kg
Mean Absolute Error (MAE)	2.15
Root Mean Squared Error (RMSE)	2.27
Coefficient of determination ((R^2))	0.88

The mean predicted price was lower than the mean actual price by approximately Rs. 1.95 per kilogram. The recorded MAE of 2.15 indicates an average absolute prediction error of approximately Rs. 2.15 per kilogram, assuming the evaluation values are

expressed in the same price units. The RMSE of 2.27 gives greater weight to larger prediction errors than the MAE.

The recorded (R^2) value of 0.88 indicates explanatory performance on the evaluated dataset. Although the recorded MAPE provides a percentage-based measure of error, the model's usefulness must be assessed in the context of the data-generation process and the intended application.

An important limitation is that the dataset was synthetically generated and the district clients were simulated within the implementation. Consequently, these measurements describe the model's behavior on the evaluated data, they do not demonstrate accuracy on real paddy transactions or establish that the model generalizes to actual market conditions.

6.1.2 RAG Assistant Results

The RAG assistant combines semantic retrieval with language generation to answer marketplace-related and agricultural questions. Its knowledge base contains 32 curated records, including 15 public agricultural or Paddy Marketing Board-related records and 17 marketplace records.

The implementation demonstrates a retrieval-based question-answering workflow in which multilingual text embeddings are compared against a FAISS index, relevant records are selected, and the retrieved context is provided to the language model.

However, the number of indexed records and the successful execution of the pipeline do not independently establish answer accuracy. A complete evaluation should record representative questions, retrieved records, answer relevance, factual support, and cases where the available information is insufficient. Quantitative performance claims should only be added when these results have been measured.

6.1.3 Marketplace Matching and Negotiation Results

The matching function applies predefined weights to district compatibility, paddy type, price closeness, and quantity compatibility. Its output is an interpretable score out of 100 that supports the ranking of potentially compatible harvest and demand listings.

The negotiation function uses two role-specific agents and validation rules to support sequential offer evaluation. The process has a maximum of six rounds and includes checks for offer constraints and termination conditions.

These implementations demonstrate how rule-based matching and language-model-assisted negotiation can be combined within one marketplace workflow. Nevertheless, successful execution of these workflows does not by itself demonstrate that the highest-ranked matches lead to successful trades or that the agents consistently produce economically beneficial agreements. Such conclusions require recorded test scenarios and appropriate evaluation metrics.

6.2 Product and technical results

The principal product result is a mobile marketplace prototype that brings several farmer-miller coordination functions together within a common application.

6.2.1 Implemented Product Functions

The C03 implementation includes the following functional areas:

- **User access and role-based interfaces:** Farmer and miller accounts support access to the relevant marketplace functions.
- **Listing management:** Farmers can manage harvest listings, while millers can publish and manage purchasing demands.
- **Marketplace matching:** The system compares eligible harvest and demand listings and ranks potential matches using the weighted scoring algorithm.
- **Price-reference support:** The price-prediction module provides an estimated value that can be considered during marketplace decisions.
- **Knowledge assistance:** The RAG assistant retrieves information from a curated knowledge base and generates contextual responses.
- **AI-assisted negotiation:** Separate farmer and miller agents exchange proposed actions under configured validation rules.

- **Partner coordination:** Connections, match selections, notifications, and contact requests support communication between potential trading partners.
- **Transaction status management:** Listing status can be updated manually to distinguish an agreement from a completed transaction.
- **Multilingual interface:** The application includes English and Sinhala language support.

These functions represent the implemented scope of the C03 prototype. Their availability should not be interpreted as evidence that every function has passed comprehensive testing or that the application is ready for large-scale operational use.

6.2.2 Technical Results

The implementation separates the mobile interface, main backend, database, and AI-related services. React Native and Expo provide the mobile client, Express handles the primary application API, MongoDB stores marketplace records, and Python supports the price-prediction and RAG functions. The negotiation functionality is provided through a separate FastAPI service.

This separation establishes distinct technical responsibilities and enables individual modules to be inspected and tested independently. It also introduces dependencies that must be managed, including service availability, network configuration, API credentials, model runtime requirements, and consistent request and response formats.

The matching implementation is rule-based rather than a trained machine-learning model. The price-reference process uses a simplified federated-learning-style simulation, and the negotiation process uses sequential language-model calls with validation rules. Clearly distinguishing these approaches is important because they have different assumptions, capabilities, and evaluation requirements.

Overall, the technical result is a modular research prototype that integrates multiple decision-support approaches within a farmer–miller marketplace. Its effectiveness beyond the demonstrated implementation remains dependent on further evaluation with representative data and realistic usage scenarios.

6.3 User or stakeholder validation

User and stakeholder validation is important for determining whether the marketplace is understandable, relevant, and suitable for its intended users. In the Sri Lankan paddy supply chain, potential stakeholders include paddy farmers, rice millers, and other parties involved in agricultural information and market coordination.

The C03 prototype provides functions intended to support these stakeholders, including listing management, matching, price-reference information, contextual assistance, and negotiation. However, technical functionality alone cannot establish that farmers and millers find the system easy to use, trust its recommendations, or consider it useful in actual trading situations.

Where formal user testing, interviews, questionnaires, or demonstrations have been conducted, the results should be reported using the actual participant numbers, procedures, and collected feedback. Relevant evaluation areas include ease of navigation, clarity of listing information, usefulness of matching results, understanding of predicted prices, quality of RAG responses, and confidence in the negotiation workflow.

If no formal stakeholder evaluation has been completed, this section should explicitly state that user acceptance has not yet been empirically established. Informal feedback or a demonstration should not be presented as a formal usability study.

A suitable next step would be to conduct structured testing with representative farmers and millers, subject to appropriate consent and research procedures. Such testing could combine task-completion observations with a questionnaire or short interview. The findings would help identify usability problems and requirements that are not evident through technical testing alone.

6.4 Integration results

Integration results concern the extent to which the internal modules of marketplace component operate together and the degree to which the component exchanges information with the wider Digital Goviya solution.

6.4.1 Internal C03 Integration

The Marketplace component architecture connects the mobile application to the Express backend through HTTP API requests. The backend coordinates marketplace operations and accesses MongoDB for persistent records. AI-related requests are directed to the appropriate Python module or service.

The RAG workflow combines embedding generation, FAISS retrieval, and language-model generation. The price-reference module supplies estimated prices, while the negotiation service coordinates the two agents and applies validation rules to their proposed actions.

At the application level, listing management, matching, match selection, negotiation, and manual transaction-status updates form related stages of the marketplace workflow. Separating these stages is important because a potential match does not guarantee an agreement, and an agreement does not establish that a physical transaction has been completed.

The implemented architecture therefore provides the mechanisms required for internal marketplace communication. The reliability of each integration should be confirmed through executed tests covering valid requests, invalid responses, service failures, and expected state transitions.

6.5 Summary of findings

The findings indicate that C03 integrates marketplace management with several forms of AI-assisted decision support within a modular application architecture. Its implemented functions include farmer and miller listings, rule-based matching, a price-reference model developed using synthetic data, a retrieval-augmented knowledge assistant, and a multi-agent negotiation workflow.

The price-prediction evaluation provides a quantitative baseline for assessing the performance of the current model. However, these results do not establish real-world forecasting performance because the training process relies on synthetic data and simulated district clients. Further evaluation using representative real-world

agricultural market data is required to determine the model's practical reliability and generalisability.

The matching and negotiation modules demonstrate complementary approaches to marketplace decision support. The matching mechanism provides a transparent compatibility score, while the negotiation workflow uses role-specific language-model agents constrained by validation rules. The RAG assistant combines semantic retrieval with language generation using a curated knowledge base. Although these functions support informed decision-making, their outputs remain subject to limitations associated with data quality, model reliability, and the scope of the available knowledge.

The implementation also distinguishes between reaching an agreement and completing a transaction through support for manual sold or fulfilled status updates. This design avoids treating an AI-generated agreement as confirmation that a physical trade has occurred and preserves the distinction between automated decision support and actual marketplace transactions.

Finally, technical results alone are insufficient to establish stakeholder acceptance, practical trading benefits, or successful integration across the Digital Goviya ecosystem. These aspects require appropriate testing evidence and, where applicable, formal stakeholder evaluation. The findings therefore establish a foundation for further research, refinement, and validation rather than demonstrating complete real-world readiness.

7 DISCUSSION

7.1 Interpretation of findings

The C03 research investigated how AI-assisted decision-support techniques could be incorporated into a digital marketplace connecting paddy farmers and rice millers in Sri Lanka. The implemented prototype combines marketplace listing management, rule-based matching, price-reference prediction, retrieval-augmented generation (RAG), and multi-agent negotiation. The findings demonstrate the technical implementation of these functions, while their practical effectiveness must be interpreted in relation to the evaluation evidence and limitations.

7.1.1 Interpretation of Price-Prediction Findings

The price-reference model evaluation provides an indication of the prediction errors and explanatory performance of the implemented model. The relatively low coefficient of determination indicates that the model explains only a limited proportion of the variation in the evaluated target values. Therefore, the current results do not provide sufficient evidence to conclude that the model can reliably predict actual paddy market prices.

The comparison between the mean actual and predicted prices showed a relatively small difference. However, a small difference between the average values does not necessarily indicate accurate predictions for individual samples. Additional evaluation metrics provide further insight into prediction errors and the model's ability to explain variations in the target values.

The model uses synthetic training data and simulated district clients. Consequently, the results primarily demonstrate the behavior of the implemented prediction and parameter-aggregation workflow rather than its effectiveness under real-world agricultural market conditions. Further evaluation using representative real-world data and appropriate validation procedures would be necessary before the predictions could be considered reliable for practical market decisions.

7.1.2 Interpretation of Matching and Negotiation Findings

The matching module uses a weighted scoring algorithm to rank harvest and demand listings according to district, paddy type, price closeness, and quantity compatibility. Its principal advantage is transparency: the score can be traced to predefined rules rather than an opaque learned model.

Nevertheless, a high score indicates compatibility according to the selected criteria, not a confirmed probability of a successful transaction. Other factors, including grain quality, moisture content, transportation distance, timing, and mill capacity, may influence actual trading decisions but are not fully represented in the current scoring formula.

The negotiation module extends the marketplace beyond listing discovery by introducing separate farmer and miller agents. Their proposed actions are processed sequentially and checked against configured constraints. This demonstrates a structured way to incorporate language-model reasoning into a negotiation workflow. However, successful execution alone does not establish that the agents consistently reach fair, optimal, or commercially beneficial agreements.

7.1.3 Interpretation of RAG Findings

The RAG assistant combines semantic retrieval with language generation to provide contextual responses based on a curated knowledge base. This design offers a way to connect user questions with stored marketplace and agricultural information rather than relying exclusively on the language model's general knowledge.

Its usefulness depends on the relevance, accuracy, and coverage of the indexed records. If the knowledge base does not contain the information required to answer a question, the assistant may provide incomplete or insufficiently supported responses. Therefore, the implementation demonstrates a retrieval-based assistance mechanism, but its factual reliability and usefulness to farmers require systematic evaluation.

Overall, the findings suggest that the different techniques can be brought together within one marketplace prototype. They do not yet establish that the combined system improves actual trading outcomes or resolves the wider challenges of the Sri Lankan paddy supply chain.

7.2 Comparison with literature baselines and existing solutions

The marketplace design can be compared with established research approaches in digital agriculture, retrieval-augmented generation, federated learning, and multi-agent systems. These comparisons help explain the design choices and identify where the implementation follows established concepts and where additional evaluation is required.

7.2.1 Digital Agriculture and Marketplace Coordination

Wolfert et al. (2017) discuss the role of big data and digital technologies in agriculture, including their potential to support decision-making and improve information use across agricultural systems. This broader research context supports the motivation for investigating digital tools that connect agricultural information with operational decisions.

Marketplace applies this general direction to a specific marketplace setting by combining harvest and demand listings, compatibility-based matching, price-reference information, and negotiation support. Unlike a standalone information service, the prototype attempts to connect several stages of potential trading within one application.

However, this architectural difference does not establish that marketplace performs better than existing agricultural marketplaces. A valid comparative evaluation would require selected systems, verified information about their features, consistent evaluation criteria, and evidence collected under comparable conditions. The current implementation does not provide such a controlled comparison.

7.2.2 Retrieval-Augmented Generation

Lewis et al. (2020) introduced retrieval-augmented generation as an approach that combines a language model with retrieved external knowledge. The marketplace assistant follows this general principle by retrieving relevant records from a FAISS index and providing them as context to a language model.

The implementation adapts this approach to a curated marketplace knowledge base containing agricultural and application-specific information. Multilingual embeddings and language-oriented preprocessing are used to support English, Sinhala, and Singlish queries.

The similarity is in the retrieval-and-generation architecture rather than in proven performance. The current C03 implementation has not been shown to outperform a language model without retrieval or another question-answering baseline. A controlled experiment comparing answer relevance, factual support, retrieval quality, and unsupported statements would be needed to establish the value added by retrieval.

7.2.3 Federated Learning and Price Prediction

McMahan et al. (2017) presented Federated Averaging (FedAvg), an algorithm for aggregating locally computed model updates. Kairouz et al. (2021) reviewed the broader federated-learning research landscape, including its methods, challenges, and open problems.

C03 uses a simplified federated-learning-style process in which four simulated district clients train an SGDRegressor model and contribute coefficients and intercepts to an averaging procedure. This provides an opportunity to investigate the basic idea of combining model parameters across simulated data partitions.

However, the implementation should not be equated with a complete distributed federated-learning system. The clients operate within a simulation, the training dataset is synthetic, and the aggregation procedure does not by itself provide formal privacy guarantees. Furthermore, averaging model parameters is not necessarily equivalent to reproducing the standard FedAvg procedure in every detail. The implementation should therefore be described according to its actual algorithm and update rules.

The recorded evaluation results provide an initial performance baseline for the current model. A stronger comparison would evaluate alternative models or aggregation methods using the same dataset splits, preprocessing, evaluation samples, and metrics. Without such an experiment, it is not possible to conclude that the current approach outperforms centralized learning or alternative federated-learning algorithms.

7.2.4 Multi-Agent Negotiation

Wooldridge (2009) provides a foundation for understanding intelligent agents and multi-agent systems. The marketplace negotiation workflow draws on the general concept of representing different decision-making roles through separate agents.

The implementation assigns distinct prompts and negotiating constraints to a Farmer Agent and a Miller Agent. Their sequential decisions are coordinated by the negotiation service, while validation rules check proposed actions and limit the number of rounds.

This is a practical prototype of role-based agent interaction, but it differs from a fully autonomous or independently distributed multi-agent system. The two agents use sequential language-model calls and rely on an orchestrating service. Their operation therefore demonstrates a structured negotiation workflow rather than establishing strategic optimality, economic fairness, or reliable real-world bargaining outcomes.

7.2.5 Overall Comparison

The comparison indicates that marketplace draws on established ideas but adapts them to a specific marketplace prototype.

Research area	**Established concept**	**C03 implementation**	**Evidence still required**
Digital agriculture	Digital information and data-supported decision-making	Farmer–miller listings and marketplace coordination	Evidence of improved real-world coordination
RAG	Retrieval combined with language generation	FAISS retrieval and contextual language-model responses	Comparative answer-quality evaluation
Federated learning	Local model training and	Four simulated clients and a simplified averaging process	Comparison against suitable baselines

	parameter aggregation		and real-data evaluation
Multi-agent systems	Distinct agents with different roles	Sequential farmer and miller language-model agents	Negotiation-quality and outcome evaluation
Marketplace matching	Matching based on compatibility criteria	Transparent weighted scoring rules	Validation against real trading outcomes

Table 9 : Overall Comparison

The comparison supports the rationale behind the selected design approaches. It does not establish superiority over existing systems, since the current research has not conducted a comprehensive benchmark or controlled comparative evaluation.

7.3 Research contribution

The research contribution of Marketplace component lies in the design and implementation of a prototype that brings multiple decision-support mechanisms into a farmer-miller marketplace for the Sri Lankan paddy supply-chain context.

First, the component demonstrates how listing management and transparent compatibility-based matching can be combined with price-reference support. The matching algorithm applies explicit criteria to rank potential harvest-demand pairs, making its decisions relatively straightforward to inspect and revise.

Second, the research demonstrates a RAG-based information-assistance workflow that combines multilingual embeddings, vector retrieval, a curated knowledge base, and language generation. This provides a technical foundation for exploring contextual assistance within an agricultural marketplace.

Third, the price-reference module demonstrates a simplified federated-learning-style parameter-aggregation workflow using simulated district clients. Although the current

experiment does not establish real-world forecasting performance or formal privacy protection, it provides an implementation that can be extended and evaluated in subsequent research.

Fourth, the negotiation module demonstrates how two role-specific language-model agents can be coordinated through a service that validates their proposed actions against predefined constraints. The implementation distinguishes agreement generation from transaction completion, providing a clearer representation of the marketplace lifecycle.

Finally, the research contribution includes the integration of these functions into one modular application. Rather than evaluating each function only as an isolated algorithm, the prototype explores how the different capabilities can support related stages of marketplace interaction.

The contribution should therefore be understood as an implemented research prototype and a basis for further evaluation. Claims that the system increases farmer income, improves market efficiency, guarantees fair prices, or provides formal privacy protection would require additional evidence.

7.4 Product contribution

The product contribution of coponent is a mobile marketplace prototype designed around the interaction between paddy farmers and rice millers.

The application provides separate farmer and miller workflows for publishing and managing harvest and demand listings. Its matching function identifies potentially compatible listings, while partner connections and match selections support the progression from discovery to negotiation. Notifications and contact-request functionality provide additional coordination mechanisms.

The price-reference module supplies an estimated value that users can consider when assessing offers. The RAG assistant provides a channel for obtaining contextual information from the indexed knowledge base, and the negotiation workflow allows

two role-specific agents to generate and evaluate proposed offers under configured constraints.

The product also distinguishes between reaching an agreement and completing a transaction. Users can manually update listings to reflect sold or fulfilled outcomes, rather than allowing an agent-generated agreement to automatically imply that the physical trade has occurred.

The modular architecture is another product-level contribution. The separation of the mobile application, Express backend, database, Python AI modules, and FastAPI negotiation service provides identifiable boundaries for maintenance and future extension.

These contributions describe the functionality and architecture of the prototype. Their practical value to farmers and millers remains to be established through user evaluation, realistic trading scenarios, and deployment under suitable operational conditions.

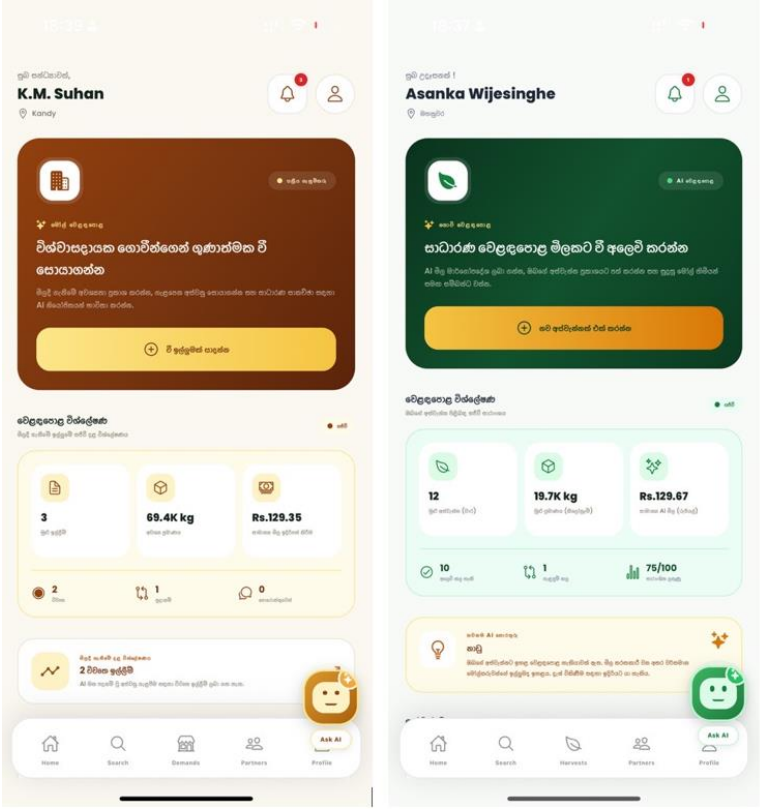


Figure 14 : Farmer and Miller Platforms of Digital Goviya

7.5 Limitations and threats to validity

Several limitations affect the interpretation and generalizability of the findings.

7.5.1 Data Limitations

The price-prediction model was developed using synthetic data generated through a predefined formula and added noise. Although this enables repeatable experiments, it may not capture the variability, seasonality, quality differences, regional influences, and market dynamics present in real paddy transactions.

The four district clients are simulated within the implementation. Therefore, the current experiment does not demonstrate that separate institutions or devices can train and exchange model updates in a real distributed environment.

The RAG assistant also depends on a limited curated knowledge base of 32 records. Its answers may be incomplete when questions concern information that is missing, outdated, or insufficiently represented in the index.

7.5.2 Evaluation Limitations

The available price-prediction metrics provide an initial quantitative assessment, but the recorded (R^2) value of 0.88 indicates limited explanatory performance on the evaluated data. The results should not be interpreted as proof of practical forecasting accuracy.

Similarly, the matching algorithm has not been established as a predictor of successful transactions. Its scoring rules are based on selected compatibility criteria, and its rankings may not capture important factors that affect real trading decisions.

The negotiation workflow has defined validation and termination rules, but its implementation alone does not establish that it produces fair or economically optimal agreements. A systematic evaluation would require a documented collection of scenarios and clearly defined success criteria.

The extent of stakeholder validation also affects the conclusions. If formal user studies have not been completed, usability, acceptance, and practical usefulness remain unverified.

7.5.3 Design and Scope Limitations

The matching score does not currently incorporate every factor that may influence compatibility, such as GPS distance, grain quality, moisture content, harvest timing, or mill capacity. Fixed weights and thresholds provide transparency but may require revision when tested against real trading situations.

The negotiation service uses sequential language-model calls rather than independent agents operating across distributed environments. Its behavior depends on model responses, prompts, validation rules, and the configured constraints.

The RAG assistant does not continuously crawl live public sources. Its usefulness for time-sensitive agricultural information therefore depends on how the curated knowledge base is maintained.

Furthermore, the application-level handling of negotiation limits does not automatically guarantee that those limits remain confidential in storage or across every service boundary. Formal privacy guarantees would require additional mechanisms and evaluation.

7.5.4 Integration and Deployment Limitations

The internal marketplace architecture includes multiple services with separate configuration and runtime requirements. Their operation depends on correct network access, database connectivity, model availability, API configuration, and error handling.

Integration with the wider Digital Goviya components must be evaluated separately. A development environment also differs from a production deployment in areas such as scalability, availability, monitoring, access control, and operational support. These aspects require further verification before the system can be considered suitable for wider use.

7.5.5 Threats to Validity

The principal threats to validity are summarized below.

Threat	Potential effect	Mitigation or future action
Synthetic training data	Limits the relevance of price-model results to actual markets.	Evaluate with representative real-world data when available.
Simulated district clients	Limits claims about distributed federated learning.	Test with independently operated clients and documented aggregation procedures.
Limited knowledge base	May reduce retrieval coverage and answer completeness.	Expand and maintain curated sources and evaluate retrieval quality.
Rule-based matching	May not reflect all factors influencing trading outcomes.	Validate against representative matching scenarios and observed transactions.
Limited stakeholder evidence	Prevents strong conclusions about usability and acceptance.	Conduct structured user evaluation with appropriate consent.
Lack of controlled baselines	Prevents claims of superiority over alternative methods.	Compare against clearly defined baselines using consistent evaluation procedures.
Incomplete group integration	Limits claims about end-to-end ecosystem functionality.	Verify interfaces and conduct cross-component integration tests.

Table 10 : Table of Treats

These limitations do not remove the value of the implementation. Instead, they define the scope within which the current findings can be interpreted and identify the evidence needed to support stronger conclusions.

7.6 Implications

The findings have implications for future research, marketplace product development, and the potential application of digital decision-support systems in Sri Lanka's paddy supply chain.

7.6.1 Research Implications

The prototype provides a foundation for studying how different AI approaches can be combined within an agricultural marketplace. Future research could compare the current weighted matching algorithm with alternative approaches, evaluate RAG against a model-only baseline, and assess the price-reference model using representative data.

The simulated federated-learning workflow could also be extended to compare aggregation algorithms under consistent experimental conditions. Such experiments should document client data distributions, model updates, evaluation splits, and privacy assumptions to make the findings reproducible.

For negotiation, future work could assess constraint compliance, agreement rates, number of rounds, and the sensitivity of outcomes to prompts and model configurations. These measures would provide stronger evidence than successful execution alone.

7.6.2 Product and Stakeholder Implications

For a future marketplace, the combination of listings, matching, contextual information, and negotiation could provide a common interface for exploring potential trading opportunities. Transparent matching rules may help users understand why particular listings appear in their results, while price references may provide an additional point of consideration when reviewing offers.

However, these benefits remain potential outcomes rather than established impacts. Real-world use would require evaluation of data accuracy, user trust, language accessibility, connectivity, and the practical requirements of farmers and millers. Users

should also be able to distinguish estimated prices and AI-generated recommendations from official information and confirmed transactions.

7.6.3 Privacy and Responsible AI Implications

The project highlights the importance of distinguishing between application-level access controls and formal privacy-preserving computation. Simulated federated learning does not automatically guarantee confidentiality, and separate agent prompts do not ensure that negotiating limits are protected throughout storage and service communication.

Future versions should assess data minimization, secure configuration, access permissions, sensitive-record handling, and appropriate protection for information exchanged between participants. AI-generated outputs should remain subject to clear validation and human oversight, particularly where prices and trading decisions may affect users financially.

7.6.4 Future Development Implications

The next development stage should prioritize validation with representative data, systematic technical testing, stakeholder feedback, and verified integration with the other Digital Goviya components. The price-reference model requires stronger evaluation; the RAG knowledge base requires ongoing quality management; the matching algorithm requires validation against realistic compatibility scenarios; and the negotiation workflow requires systematic assessment of its outcomes.

The architecture can provide a starting point for these improvements because its major functions are separated into identifiable modules. Nevertheless, modularity alone does not guarantee scalability, reliability, or deployment readiness. These properties must be established through appropriate engineering and evaluation activities.

In summary, C03 demonstrates a technical approach to combining marketplace coordination with AI-assisted information retrieval, price referencing, matching, and negotiation. Its principal implication is the opportunity it creates for further investigation and development. Establishing practical benefits for farmers and millers will require stronger empirical evidence, broader stakeholder involvement, and reliable integration under realistic operating conditions.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

The research focused on designing and implementing an AI-powered farmer–miller marketplace as Component 03 of the Digital Goviya project, titled AI-Driven Privacy-Preserving Digital Ecosystem for Modernizing Sri Lanka's Paddy Supply Chain. The main purpose of C03 was to investigate how digital marketplace functions and AI-assisted decision-support techniques could be combined to support interactions between paddy farmers and rice millers.

The developed prototype brings together several functions, including user authentication, harvest and demand listing management, marketplace matching, price-reference prediction, retrieval-augmented generation (RAG), and multi-agent negotiation. These functions are organized within a modular architecture consisting of a React Native and Expo mobile application, a Node.js and Express backend, MongoDB persistence, and supporting Python modules and services.

The marketplace matching function applies a transparent weighted scoring algorithm to identify potentially compatible harvest and demand listings. The price-reference module demonstrates a simplified federated-learning-style process using simulated district clients, while the RAG assistant retrieves information from a curated knowledge base to support contextual question answering. The negotiation service uses separate farmer and miller agents whose proposed actions are checked against predefined constraints.

The evaluation results provide an initial basis for understanding the behavior of the implemented price-prediction model. The research also highlights the importance of distinguishing technical implementation from practical effectiveness. A matching score does not guarantee a successful trade, an AI-generated agreement does not confirm transaction completion, and retrieval-based responses are not necessarily accurate merely because relevant records were retrieved. These functions require evaluation against appropriate criteria and realistic scenarios.

In conclusion, AI-Powered Farmer-Miller Marketplace component demonstrates a technical approach to combining digital marketplace coordination with several AI-assisted capabilities in one research prototype. It provides a foundation for further investigation into agricultural marketplace systems, but additional validation using representative data, stakeholder feedback, and realistic deployment conditions is required before its practical impact can be established.

8.2 Achievement of objectives

The achievement of the C03 research objectives is assessed according to the implemented functionality and the available evaluation evidence. The following table summarizes the principal objectives and the extent to which they have been addressed.

Research objective	**Achievement and evidence**
Develop a digital marketplace for farmers and millers.	A mobile marketplace prototype was implemented with farmer and miller accounts, harvest and demand listings, and related marketplace operations.
Identify potentially compatible harvest and demand listings.	A weighted rule-based matching algorithm was implemented using district, paddy type, price closeness, and quantity compatibility.
Investigate AI-assisted price-reference generation.	A simplified federated-learning-style price-prediction workflow was implemented and evaluated using synthetic data. The recorded results provide an initial baseline but do not establish real-world accuracy.
Provide contextual agricultural and marketplace information.	A RAG assistant was implemented using multilingual embeddings, FAISS retrieval, a curated knowledge base, and a language model. Further answer-quality evaluation is required to establish its reliability.

Support structured farmer–miller negotiation.	Two role-specific language-model agents were implemented with validation rules and a bounded negotiation process. Their practical negotiation quality requires further evaluation.
Combine marketplace functions within a modular architecture.	The mobile application, Express backend, database, and AI-related services were organized into identifiable modules with defined communication paths.
Consider privacy and responsible AI use.	The design incorporates authentication, access-control considerations, and constraints on agent actions. Formal privacy guarantees and comprehensive security effectiveness have not been established by these measures alone.

Table 11 : Achievement of research objectives

The objectives concerning prototype development and the implementation of the principal marketplace functions have been addressed through the current component implementation. The objectives relating to predictive reliability, user acceptance, practical trading benefits, and privacy effectiveness require further evidence.

This distinction is important because achieving an implementation objective is not equivalent to demonstrating a real-world outcome. The research therefore provides an initial technical foundation while identifying the areas that require additional investigation.

8.3 Individual research and product contributions

The individual contribution of this research is the design, implementation, and evaluation of the AI-powered Farmer–Miller Marketplace within the wider Digital Goviya project. While the overall project includes four components, this thesis focuses on the marketplace component and its specific technical and research outcomes.

8.3.1 Individual Research Contribution

The research contribution consists of investigating how multiple decision-support approaches can be combined within a farmer–miller marketplace. These approaches include transparent rule-based matching, a simplified federated-learning-style price-reference workflow, RAG-based knowledge assistance, and role-specific AI-assisted negotiation.

The research also identifies the limitations associated with each approach. The price model's synthetic training data restrict the generalizability of its results; the matching score depends on predefined criteria; the RAG assistant depends on the coverage and quality of its knowledge base; and the negotiation agents require systematic assessment beyond successful execution.

By documenting these limitations alongside the implementation, the research establishes a basis for future experiments and provides a clearer distinction between demonstrated technical behavior and unverified practical benefits.

8.3.2 Individual Product Contribution

The principal product contribution is the marketplace prototype. Its implemented functions include:

- Farmer and miller authentication and role-oriented interfaces.
- Harvest and demand listing management.
- Rule-based matching and match selection.
- Price-reference prediction using a simplified parameter-aggregation process.
- RAG-based contextual assistance using a curated knowledge base.
- AI-assisted negotiation through farmer and miller agents.
- Partner connections, notifications, and contact-request functionality.
- Manual sold or fulfilled status updates to distinguish transaction completion from negotiation agreement.
- English and Sinhala interface support.

The product contribution also includes the modular backend architecture and the communication mechanisms connecting the application to its supporting services. This separation provides a foundation for independent testing, maintenance, and future extension.

These contributions represent the implemented scope of C03. They should not be interpreted as proof of commercial readiness, guaranteed privacy, improved farmer income, or successful integration across all four project components.

8.4 Future directions

Several improvements could extend the research and product beyond the current prototype.

8.4.1 More Realistic Federated Learning

The simulated district clients could be extended into independently operated clients that train models on separate datasets and exchange model updates through a defined coordination mechanism. Future work could compare FedAvg, FedProx, and other appropriate methods using consistent experimental settings.

Privacy should be investigated separately from parameter aggregation. Depending on the research requirements, future versions could examine secure aggregation, differential privacy, and protection against information leakage from model updates. Any claimed privacy guarantee would need to be supported by an explicit mechanism and evaluation.

8.4.2 Improved Matching and Recommendation

The matching algorithm could be extended to include additional compatibility factors, such as transportation distance, grain quality, moisture content, harvest timing, delivery constraints, and mill capacity. The weights and recommendation thresholds should be evaluated using representative scenarios or observed trading outcomes rather than being treated as universally appropriate values.

Where sufficient reliable data become available, the current rule-based approach could be compared with data-driven ranking methods. The goal would be to determine whether more complex techniques provide measurable benefits while retaining sufficient transparency for users.

8.4.3 Improved RAG Accuracy and Knowledge Coverage

The knowledge base could be expanded with verified agricultural information, official price information where available, and additional marketplace guidance. A documented update process would help keep the information accurate and relevant.

The RAG assistant should also be evaluated using a representative collection of English, Sinhala, and Singlish questions. Future work could measure retrieval relevance, answer support, response completeness, and the frequency of unsupported statements. Additional safeguards could encourage the assistant to acknowledge when reliable information is unavailable rather than generating an uncertain answer.

8.4.4 Advanced Negotiation Evaluation

The negotiation workflow could be evaluated using a structured set of scenarios covering compatible offers, incompatible constraints, repeated counteroffers, invalid proposals, and unsuccessful negotiations. Relevant metrics could include constraint-compliance rate, agreement rate, average number of rounds, termination behavior, and consistency across repeated runs.

Further development could explore alternative language models, prompt strategies, or negotiation policies. These approaches should be compared using clearly defined criteria rather than assuming that a more complex model will necessarily produce better outcomes. Privacy protection for negotiating limits should also be strengthened and evaluated across storage, API communication, and service boundaries.

8.4.5 Stakeholder Validation and Usability

Future work should involve structured evaluation with representative farmers and millers, subject to appropriate consent and research procedures. Participants could be asked to complete realistic tasks, such as publishing a listing, reviewing matches, asking a marketplace question, and examining a negotiation outcome.

Task-completion observations, participant feedback, and usability measures could be used to identify interface problems and determine whether the functions meet user needs. Such a study would also help assess the accessibility of the Sinhala and English interfaces and the clarity of AI-generated recommendations.

8.4.8 Long-Term Research Direction

The longer-term direction is to evolve Marketplace component from a research prototype into a more thoroughly evaluated marketplace system. This would require stronger evidence of predictive accuracy, more realistic matching criteria, reliable knowledge assistance, well-tested negotiation behavior, verified privacy controls, and feedback from intended users.

The four Digital Goviya components could eventually be evaluated as a coordinated ecosystem, provided that their interfaces and data exchanges are implemented and tested. Such an evaluation could examine whether the combined solution provides practical value beyond the functions of its individual components.

Ultimately, the future of the marketplace depends not only on adding more AI techniques but also on demonstrating that its outputs are accurate enough for their intended use, understandable to stakeholders, secure in operation, and useful under realistic agricultural conditions.

REFERENCES

[1] P. Kairouz et al., "Advances and open problems in federated learning," Foundations and Trends in Machine Learning, vol. 14, nos. 1–2, pp. 1–210, 2021, doi: 10.1561/2200000083.

https://arxiv.org/abs/1912.04977?utm_source=chatgpt.com

[2] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020. [Online]. Available: https://mlanthology.org/neurips/2020/lewis2020neurips-retrievalaugmented/

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, vol. 54, pp. 1273–1282, 2017. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[4] M. Wooldridge, An Introduction to Multiagent Systems, 2nd ed. Chichester, U.K.: John Wiley & Sons, 2009.

[5] J. Wolfert, L. Ge, C. Verdouw, and M. J. Bogaardt, "Big data in smart farming – A review," Agricultural Systems, vol. 153, pp. 69–80, 2017, doi: 10.1016/j.agsy.2017.01.023.

https://www.sciencedirect.com/science/article/pii/S0308521X16303754?via%3Dihub

GLOSSARY

AI-assisted negotiation: A process in which AI agents support the exchange of offers and counteroffers between two parties while operating within defined rules and constraints.

Embedding: A numerical representation of text that allows semantically related content to be compared by similarity.

Federated learning: A machine-learning approach in which model training can occur across multiple clients, with model updates combined instead of requiring all training data to be centrally collected.

Farmer–miller marketplace: A digital platform intended to help paddy farmers and rice millers discover compatible trading opportunities and coordinate potential transactions.

FAISS index: A data structure used to search vector representations efficiently according to similarity.

Large language model (LLM): An AI model trained on large amounts of text that can generate and interpret natural-language responses.

Matching score: A numerical value used by the marketplace to rank potential farmer–miller opportunities according to predefined compatibility criteria.

Negotiation agent: A software agent that represents a party's position during a structured negotiation process.

Price reference: An indicative price estimate intended to support decision-making; it is not a guaranteed market price or a binding transaction price.

Retrieval-augmented generation (RAG): An approach that retrieves relevant information from a knowledge source and uses that information to support a language model's response.

Synthetic dataset: Data generated artificially for development or evaluation rather than collected directly from real-world observations.

Similarity threshold: A minimum similarity score used to determine whether retrieved content is sufficiently relevant for a particular query.

Transaction status: The recorded state of a marketplace listing or transaction. An agreed negotiation and a completed or fulfilled transaction represent different stages.

APPENDICES

Appendix A Research instruments or extended results

Software testing evidence is included to demonstrate the implemented functionality of the marketplace component. The evidence may cover user authentication, harvest and demand management, matching operations, multilingual retrieval-augmented generation (RAG) responses, negotiation workflows, and API request handling.

The following screenshots and outputs can be included as supporting evidence:

- Farmer and miller registration and login.
- Creation and management of farmer harvest listings and miller demand listings.
- Marketplace matching results with compatibility scores.
- RAG assistant queries and responses in supported languages.
- Negotiation histories containing offers, counteroffers, validation outcomes, and final negotiation statuses.
- API testing outputs showing successful requests, invalid inputs, and error handling.
- Relevant evaluation logs or output files supporting the reported model metrics.

For each test, the test identifier, input data, expected result, actual result, execution status, and execution date should be recorded where applicable. All evidence should be reviewed to ensure that it does not expose personal information, passwords, access tokens, API keys, or other confidential information.

Category	Executed / Pass	Fail	Not Tested (Scope)
Unit Testing	47	0	FAISS/OpenAI retrieval, Ollama multi-round negotiation
Integration Testing	6 JWT checks + 1 FL pipeline check	1	Matching / RAG / negotiation happy paths, sold-exclusion flow
NFR – Performance	Auth-reject latency + FL predict latency measured	—	Authenticated happy-path latency (login, harvest create, RAG, negotiation)
Regression Testing	5 historical fixes re-checked via source/unit tests	0	Live UI regression (matching tabs, Mark as Sold, connected partners)

Unit Testing

Summary

Suite	Tests	Pass	Fail
Matching – evaluateDemand	14	14	0
RAG language / normalize / fallback filter	10	10	0
FL prediction.py CLI	5	5	0
Negotiation DecisionValidator + schema	10	10	0
Auth middleware + private-price schema	8	8	0
TOTAL EXECUTED	**47**	**47**	**0**

Test Cases

Test ID	Area	Result	Evidence
UT-RAG-06	Sinhala query normalization	PASS	unit-run-log.txt
UT-RAG-07	Singlish query normalization	PASS	unit-run-log.txt
UT-RAG-08	contains_sinhala	PASS	unit-run-log.txt
UT-RAG-09	Retrieve fallback rule (scores < 0.20 keep top 3)	PASS	unit-run-log.txt
UT-RAG-10	Threshold scores are kept	PASS	unit-run-log.txt
UT-FL-01	Missing CLI args → error JSON	PASS	unit-run-log.txt
UT-FL-02	Model files exist	PASS	unit-run-log.txt
UT-FL-03	Valid input returns predictedPrice	PASS	unit-run-log.txt
UT-FL-04	Invalid district → error JSON	PASS	unit-run-log.txt
UT-FL-05	Non-numeric quantity fails	PASS	unit-run-log.txt
UT-NEG-01	Farmer cannot accept below minimum	PASS	unit-run-log.txt
UT-NEG-02	Farmer can accept at/above minimum	PASS	unit-run-log.txt
UT-NEG-03	Farmer counter below minimum invalid	PASS	unit-run-log.txt
UT-NEG-04	Farmer valid counter accepted	PASS	unit-run-log.txt
UT-NEG-05	Miller cannot accept above maximum	PASS	unit-run-log.txt

Appendix B Academic integrity and AI use disclosure

B.1 Purpose of AI-tool use

Generative AI tools were used as supplementary support during the preparation of the research report and the organization of technical explanations. Potential uses included improving the clarity of academic writing, structuring chapter content, explaining software architecture concepts, and assisting with the presentation of implementation and evaluation details.

B.2 Verification and researcher responsibility

The researcher remains responsible for the accuracy, originality, and integrity of the submitted work. AI-generated suggestions were not treated as independent evidence of implementation success, experimental performance, or stakeholder acceptance. Technical descriptions and quantitative results must be checked against the source code, saved

evaluation outputs, and executed tests. References must be checked against the original publications, and any externally derived ideas or information must be cited appropriately.

B.3 Representative AI-assisted workflows

Representative workflows may include requesting help to organize thesis chapters, clarify the distinction between rule-based matching and machine-learning prediction, explain retrieval-augmented generation and federated learning, and improve the presentation of model evaluation results. The actual prompts and resulting changes should be recorded where required by the university.

B.4 Scope of AI assistance

The researcher is responsible for the research design, implementation decisions, interpretation of results, and final submitted content. Any AI-assisted content was reviewed and adapted for consistency with the individual C03 marketplace implementation. This disclosure should be revised to reflect the actual extent of AI assistance and any specific disclosure requirements issued by the university.